



LxCameraSDK(C/C++)

Developer Guide

Version: V1.0

Release Date: 2026.06.04

GitHub: <https://github.com/Lanxin-MRDVS/CameraSDK>

Knowledge Base: <https://hub.mrdvs.cn/>

Official Website: <https://www.mrdvs.cn/>&<https://mrdvs.com/>

Zhejiang MRDVS Technology Co., Ltd. All Rights Reserved



Contents

1. Overview.....	1
2. Environment Setup.....	2
2.1 Recommended Host System Configuration	2
2.2 Hardware Environment Setup	2
2.3 Network Environment Setup.....	3
2.4 Software Environment Setup	4
3. SDK Directory and Project Configuration	5
3.1 LxCameraSDK Directory Structure	5
3.2 Project Configuration.....	5
3.3 CMake Project Configuration	5
3.4 Makefile Project Configuration.....	6
3.5 Runtime Libraries and Deployment	6
4. Development Workflow.....	7
4.1 General Rules for Interface Calls	7
4.2 Synchronous Interface Call Workflow.....	8
4.3 Asynchronous Interface Call Workflow	9
4.4 Multi-Camera Development Workflow	9
4.5 Built-in Application Algorithm Workflow	10
5. Notes on Interface Calls.....	10
5.1 Call Sequence and Resource Management	10
5.2 Preconditions for Some Interface Calls.....	11
5.3 Return Value Handling	12
6. Data Types and Enumerations	12
6.1 LX_STATE.....	12
6.2 LX_DATA_TYPE	13
6.3 LX_DEVICE_TYPE.....	14
6.4 LX_DEVICE_SERIALS.....	15
6.5 LX_OPEN_MODE	15
6.6 LX_2D_OUTPUT_FORMAT	16
6.7 JPEG_DECODE_METHOD	16
6.8 LX_BINNING_MODE.....	16
6.9 LX_STRUCT_LIGHT_CODE_MODE.....	16
6.10 LX_ALGORITHM_MODE.....	17
6.11 LX_CAMERA_WORK_MODE	17
6.12 LX_TRIGGER_MODE	17

6.13 LX_IO_WORK_MODE	17
6.14 LX_IO_OUTPUT_STATE	18
6.15 LX_IO_HARDWARE_MODE	18
6.16 LX_FILTER_MODE	18
6.17 LX_3D_FREQ_MODE.....	18
6.18 LX_RGBD_ALIGN_MODE	19
6.19 LX_CAN_PROTOCOL_TYPE	19
6.20 LX_POWER_MODE_TYPE.....	19
6.21 LX_CAMERA_STATUS	19
6.22 LX_DISTORTION_MODEL	20
7. Feature Parameter Description	20
7.1 Int Parameters	20
7.2 Float Parameters.....	26
7.3 Bool Parameters	27
7.4 String Parameters	28
7.5 Command Parameters	29
7.6 Ptr Parameters	29
8. Structure Description	31
8.1 Device and Parameter Information Structures	31
8.2 Frame Data Structures.....	32
8.3 Status Callback and IMU Structures	33
8.4 Point Cloud and Camera Intrinsic Structures	33
8.5 Application Algorithm Structures	34
9. API Description	35
9.1 Version, Logs, and Global Configuration	35
9.1.1 DcGetApiVersion.....	35
9.1.2 DcSetInfoOutput	36
9.1.3 DcLog.....	36
9.1.4 DcGetErrorString.....	36
9.1.5 DcSetGpuEnable / DcGetGpuEnable.....	36
9.1.6 DcSetPtpEnable / DcGetPtpEnable.....	37
9.1.7 DcSetParallelThread	37
9.1.8 DcSetJpegDecodeMethod	37
9.1.9 DcSetMaxLogFile / DcGetMaxLogFile.....	37
9.1.10 DcSetMaxLogFileSize / DcGetMaxLogFileSize.....	38
9.2 Camera Discovery, Connection, and Closing	38

9.2.1 DcGetDeviceList.....	38
9.2.2 DcOpenDevice	38
9.2.3 DcCloseDevice.....	39
9.3 Starting and Stopping Streaming	39
9.3.1 DcStartStream	39
9.3.2 DcStopStream	39
9.4 Reading and Setting Camera Parameters	39
9.4.1 DcSetIntValue.....	39
9.4.2 DcGetIntValue	40
9.4.3 DcSetFloatValue	40
9.4.4 DcGetFloatValue.....	40
9.4.5 DcSetBoolValue.....	40
9.4.6 DcGetBoolValue	41
9.4.7 DcSetStringValue.....	41
9.4.8 DcGetStringValue	41
9.4.9 DcGetPtrValue	41
9.4.10 DcSetCmd	42
9.5 Special Control, ROI, IP, User Data, and Point Cloud Saving	42
9.5.1 DcSpecialControl	42
9.5.2 DcSpecialControlExtend.....	42
9.5.3 DcSetROI	42
9.5.4 DcSetCameraIp	43
9.5.5 DcWriteUserData.....	43
9.5.6 DcReadUserData.....	43
9.5.7 DcSaveXYZ	44
9.5.8 DcSetParamPath.....	44
9.6 Callback Interfaces.....	44
9.6.1 DcRegisterFrameCallback	44
9.6.2 DcUnregisterFrameCallback.....	44
9.6.3 DcRegisterCameraStatusCallback	45
9.6.4 DcUnregisterCameraStatusCallback.....	45
9.6.5 DcRegisterImuDataCallback.....	45
9.6.6 DcUnregisterImuDataCallback	45
10. Sample Program Description	46
10.1 Sample Program Index.....	46
10.2 single_camera2: Single-Camera Synchronous Streaming	46

10.3 frame_callback: Frame Callback Streaming	47
10.4 lidarpoint_callback: Obtaining LiDAR Point Cloud Through Frame Callback.....	48
10.5 imu_callback: IMU Data Callback.....	48
10.6 multi_cameras: Multi-Camera Development	48
10.7 application_obstacle_v1: Obstacle Avoidance Algorithm V1	49
10.8 application_obstacle_v2: Obstacle Avoidance Algorithm V2	49
10.9 application_pallet: Pallet Docking Algorithm.....	49
10.10 application_location: Visual Localization Algorithm	50
11. Debugging and FAQs.....	50
12. Version and Compatibility Notes.....	51

1. Overview

This document is intended for C/C++ developers who use LxCameraSDK to connect MRDVS/LxCamera series cameras. It describes the development methods for device discovery, connection, parameter configuration, synchronous streaming, asynchronous callbacks, point clouds, IMU, multi-camera usage, built-in obstacle avoidance, pallet docking, visual localization, and other functions.

LxCameraSDK provides unified interfaces for device access control and parameter configuration. It supports Windows x86/x64, Linux x86, and Linux Arm platforms. Image data can be obtained through synchronous refresh or callbacks. Supported data types include depth images, amplitude images, 2D/RGB images, ordinary XYZ point clouds, LiDAR XYZIRT point clouds, and application algorithm outputs.

Table 1-1 SDK Capability Scope

Capability	Description
Device access	Search the device list and open a device by index, IP, SN, or ID. After the device is opened, exclusive control permission is obtained. Subsequent interfaces all depend on DcHandle.
Image streaming	Supports independent enabling and disabling of the depth stream, amplitude stream, and 2D/RGB stream. Streaming can be performed synchronously by DcSetCmd or asynchronously by registering DcRegisterFrameCallback.
Point cloud output	Supports ordinary XYZ point clouds and LxPointCloudData LiDAR point clouds. Point clouds can be saved directly in txt, ply, or pcd format.
Parameter configuration	Feature enumerations use the LX_INT, LX_FLOAT, LX_BOOL, LX_STRING, LX_CMD, and LX_PTR prefixes to distinguish read/write interfaces.
Multi-camera	Multiple cameras can be opened by IP and streamed in separate threads. Status callbacks can be registered to monitor disconnection, reconnection, and streaming status.
Built-in algorithms	Supports obstacle avoidance V1, obstacle avoidance V2, pallet docking, visual localization, and application platform custom data modes. Algorithm output is obtained through FrameInfo.app_data or LX_PTR_ALGORITHM_OUTPUT.
Extended sensors	Supports IMU data callbacks, LiDAR point cloud structures, PTP time synchronization source service, GPU acceleration switch, JPEG decoding method, and parallel computing thread configuration.

Table 1-2 Current API Version

Item	Content
API version macro	LX_CAMERA_API_VERSION V2.4.60,20260126

Item	Content
Header file directory	SDK/include/lx_camera_api.h, lx_camera_define.h, lx_camera_application.h, lx_camera_api_version.h
Sample directory	Sample/C

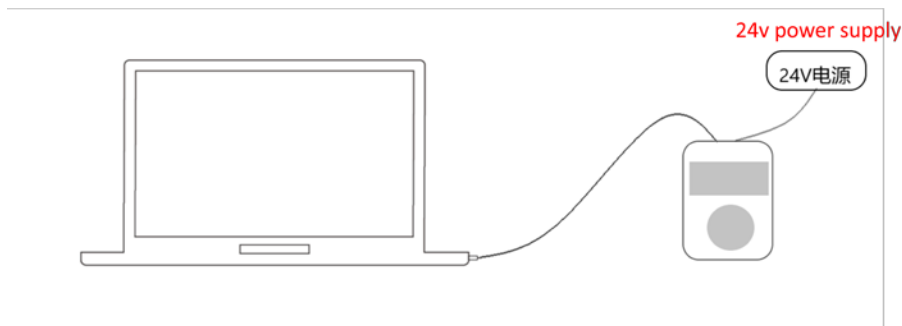
2. Environment Setup

2.1 Recommended Host System Configuration

Configuration Item	Recommended Configuration	Description
Operating system	Windows7/10/11;Ubuntu16.04 or later	For Linux Arm environments, confirm that the installation package architecture matches the runtime library architecture.
CPU	4-core Cortex-A57 or equivalent, or higher	For multi-camera use, point clouds, and algorithm outputs, a higher clock rate or more cores are recommended.
Memory	4GB or more	Reserve more memory for multi-camera display, point cloud saving, and algorithm data caching.
Network adapter	Gigabit or higher	When the depth, amplitude, and RGB streams are enabled at the same time, an independent gigabit network port is recommended. Avoid sharing it with other high-bandwidth services.
Disk	Reserve space for logs and point cloud saving	pcd/ply/txt point cloud files can quickly occupy disk space. Clean them up during debugging when needed.

2.2 Hardware Environment Setup

Connect the camera to an external 24V power supply and connect it to the host computer through a network cable, as shown in the figure. It is recommended to plan a clear IP address for each camera and configure a switch or an independent network port when necessary.



Check Item	Requirement
Power supply	Confirm that the power voltage, current, and wiring meet the camera requirements to avoid device restart or stream interruption caused by unstable power supply.
Network cable	Use a reliable network cable and confirm that the host network port negotiates to gigabit speed. The negotiated network speed can be obtained through LX_INT_LINK_SPEED.
IP planning	The default camera subnet is usually 192.168.100.*. The host network port must be configured to the same subnet and must not use a conflicting address.
Multi-camera connection	Avoid duplicate IP addresses for multiple cameras. When TOF multi-camera interference occurs, use LX_BOOL_ENABLE_MULTI_MACHINE together with on-site installation angle adjustment.

2.3 Network Environment Setup

The default camera IP subnet is 192.168.100.*. On the host computer, set the IP address of the network port connected to the camera to 192.168.100.x and the subnet mask to 255.255.255.0. If the IP has been changed, use the IP in the discovered LxDeviceInfo.ip as the reference. When connection fails, check the IP subnet, network adapter binding, firewall, and port occupation first.



When connecting the camera, it is recommended to disable the firewall to prevent camera data packets from being blocked by the host running the SDK.



Platform	Recommended Operation
Windows	Configure the IPv4 address of the network port to the same subnet as the camera. During debugging, the firewall can be temporarily disabled or network access for the SDK/sample program can be allowed.
Ubuntu	Configure the network port in the same way as on Windows. For the firewall, use <code>sudo systemctl stop ufw.service</code> , <code>sudo systemctl disable ufw.service</code> , and <code>sudo ufw status</code> to check it.
Cross-subnet	SDK discovery depends on network reachability. In cross-subnet or multi-NIC environments, first confirm the route, gateway, subnet mask, and actual camera IP.
Changing the camera IP	After <code>DcSetCameraIp</code> is used to change the IP, the device list changes. <code>DcGetDeviceList</code> must be called again before opening the device.

2.4 Software Environment Setup

Platform	Installation and Runtime Description
Windows	Run the Lanxin-MRDVS-xxx.exe installer. Use administrator privileges when installing for all users. When compiling samples, link the libraries under <code>SDK/lib/win_x64</code> or <code>win_x86</code> . At runtime, ensure that the DLLs can be found by the program.
Linux	Extract <code>Lanxin-MRDVS-xxx.tar.gz</code> , enter <code>Lanxin-MRDVS</code> , and run <code>install.sh</code> with root privileges. The installation directory is usually <code>/opt/Lanxin-MRDVS</code> .
Linux environment variables	The installation script sets <code>LD_LIBRARY_PATH</code> . For an already opened terminal, run <code>source ~/.bashrc</code> . When running as a non-root user, add <code>export LD_LIBRARY_PATH=/opt/Lanxin-MRDVS/lib:\$LD_LIBRARY_PATH</code> to the current user environment.
OpenCV	The image display and saving logic in <code>Sample/C</code> optionally uses OpenCV. OpenCV is an optional dependency. CMake detects it through

Platform	Installation and Runtime Description
	find_package(OpenCV). If found, HAS_OPENCV is defined and code such as image display and saving is enabled. If OpenCV is not found, basic acquisition and interface call logic can still be compiled and run.

3. SDK Directory and Project Configuration

3.1 LxCameraSDK Directory Structure

Directory	Content
Document	SDK documentation, developer guides, sample manuals, and FAQs.
FirmWare	Camera firmware upgrade packages.
Sample	Sample code for different languages and function scenarios. C/C++ samples are located in Sample/C. The Sample\C directory provides C++ sample projects. The samples call the C-style interfaces of LxCameraSDK and require C++11. For pure C calls, pay attention to the <code>#ifdef __cplusplus</code> branches in the header files. Some interfaces have default parameters in C++, while pure C requires all parameters to be passed explicitly.
SDK/include	C/C++ header files. Include <code>lx_camera_api.h</code> during development. Refer to <code>lx_camera_application.h</code> for application algorithm structures.
SDK/lib	Link libraries and runtime libraries for different platforms and architectures.

3.2 Project Configuration

C/C++ is recommended as the main secondary development programming language for integrating the SDK. A CMakeLists.txt script is provided to build the project using CMake. The Sample folder also provides development examples for different languages. For details, refer to the materials under Sample and MRDVS_LxCameraSDK_Linux_SampleManual_Vx.x_xxxxxxxx.

3.3 CMake Project Configuration

Sample/C/CMakeLists.txt uses C++11 and configures header and library paths separately for Windows and Linux. On Windows, it compiles and links against SDK/include and SDK/lib/win_x64 or win_x86. On Linux, it uses /opt/Lanxin-MRDVS/include and /opt/Lanxin-MRDVS/lib by default.

The current CMake configuration builds demo_single_camera2, demo_multi_cameras, demo_application_obstacle_v1, demo_application_obstacle_v2, demo_application_pallet, demo_frame_callback, demo_application_location, demo_imu_callback, and demo_lidarpoint_callback. The output directory is build/bin.

Sample Target	Source File	Main Function
single_camera2	single_camera2/single_camera2.cpp	Open a single camera synchronously, configure streams, refresh frames, and read FrameInfo, point clouds, and images.
multi_cameras	multi_cameras/multi_cameras.cpp	Multi-camera streaming by IP in multiple threads, with status callbacks and an OpenCV display queue.
frame_callback	frame_callback/frame_callback.cpp	Register a frame callback and asynchronously obtain depth, amplitude, RGB, point cloud, and algorithm output.
lidarpoint_callback	lidarpoint_callback/lidarpoint_callback.cpp	Obtain LxPointCloudData LiDAR point cloud through the frame callback and save it as ASCII PCD.
imu_callback	imu_callback/imu_callback.cpp	Register an IMU data callback, read acceleration and angular velocity data, and calculate frequency statistics.
application_obstacle_v1	application_obstacle_v1/application_obstacle_v1.cpp	Obstacle avoidance V1 mode, region index switching, and IO status query.
application_obstacle_v2	application_obstacle_v2/application_obstacle_v2.cpp	Obstacle avoidance V2 mode, reading detailed obstacle output from LxAvoidanceOutputN.
application_pallet	application_pallet/application_pallet.cpp	Pallet docking mode, reading LxPalletPose output.
application_location	application_location/application_location.cpp	Visual localization mode, map import/export, mapping/localization, odometry, relocalization, and laser input.

3.4 Makefile Project Configuration

The single_camera2 directory also provides a Makefile for quickly building the minimum synchronous streaming sample in a Linux environment. The Makefile currently uses path=/opt/MRDVS by default, and compilation uses \$(path)/include and \$(path)/lib. If the SDK is installed to /opt/Lanxin-MRDVS by the installation script, change path in the Makefile to /opt/Lanxin-MRDVS first, or change it to the actual SDK installation directory.

3.5 Runtime Libraries and Deployment

Scenario	Handling Method
Windows runtime	Add the SDK dynamic library directory to PATH, or place the required DLLs in the same directory as the executable program.
Linux runtime	The default installation path is usually /opt/Lanxin-MRDVS. At runtime, use install.sh to write environment variables, or manually set

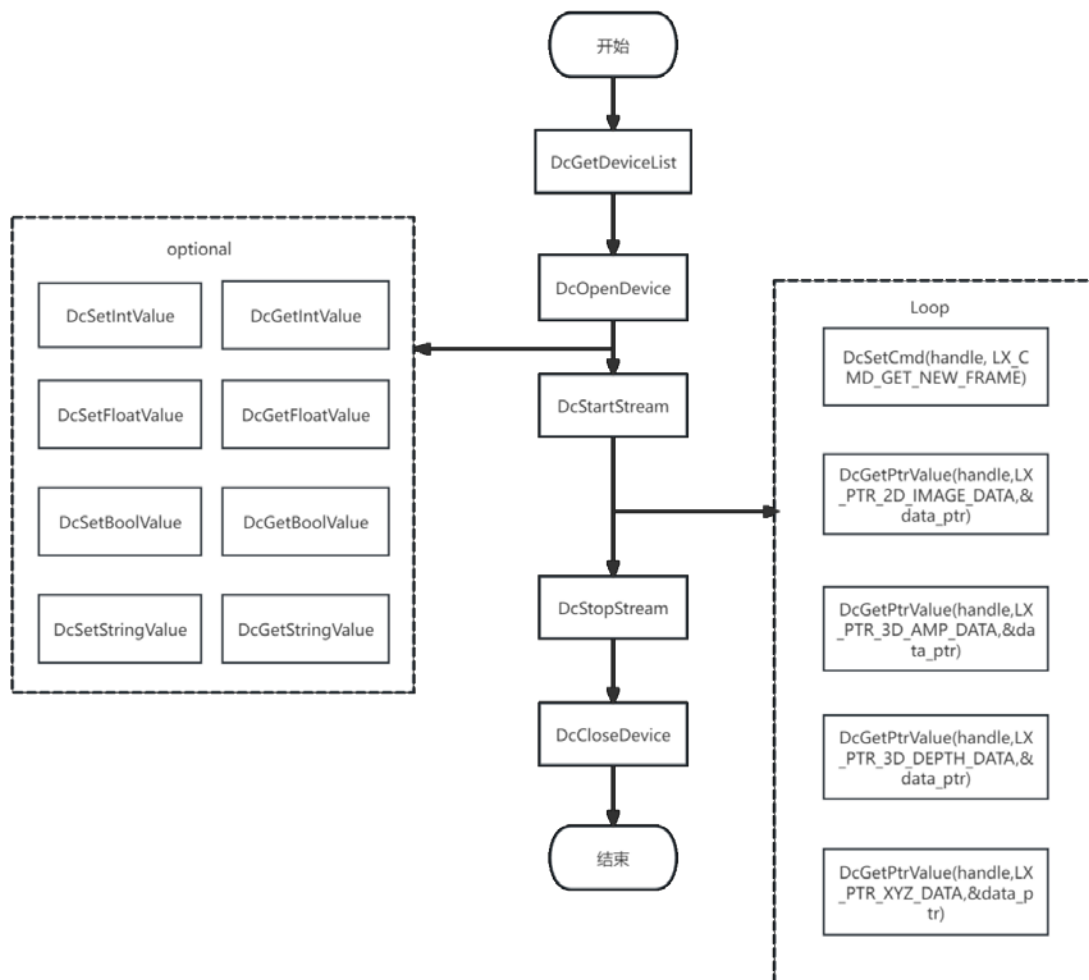
Scenario	Handling Method
	LD_LIBRARY_PATH=/opt/Lanxin-MRDVS/lib:\$LD_LIBRARY_PATH. If the standalone Makefile in single_camera2 is used and path is still /opt/MRDVS, adjust LD_LIBRARY_PATH according to the actual path as well.
Cross-platform macros	The samples use _WIN32 to distinguish Windows and Linux. Window display, keyboard detection, and thread exit logic must be handled by platform.
OpenCV optional	If OpenCV is not found during compilation, image window display code is not enabled. SDK streaming, parameter, and algorithm interfaces can still be used.

4. Development Workflow

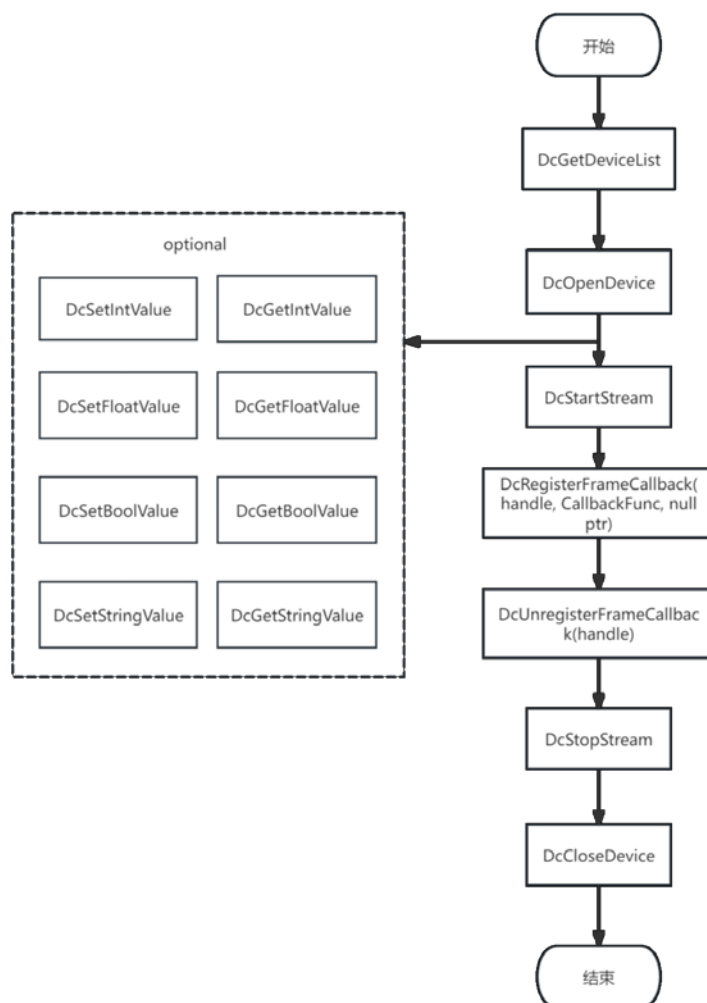
4.1 General Rules for Interface Calls

Feature Prefix	Interface	Purpose
LX_INT	DcSetIntValue / DcGetIntValue	Read and write integer parameters, such as exposure, image size, trigger mode, algorithm mode, and filter level.
LX_FLOAT	DcSetFloatValue / DcGetFloatValue	Read and write floating-point parameters, such as filter strength, light intensity, frame rate, and temperature.
LX_BOOL	DcSetBoolValue / DcGetBoolValue	Read and write Boolean switches, such as data stream switches, auto exposure, undistortion, synchronized frame, and IMU enable.
LX_STRING	DcSetStringValue / DcGetStringValue	Read and write string parameters, such as firmware name, algorithm parameters, algorithm version, parameter import/export paths, and others.
LX_CMD	DcSetCmd	Execute commands, such as refreshing frames, software trigger, device restart, and restoring default parameters.
LX_PTR	DcGetPtrValue	Obtain internally managed data pointers, such as FrameInfo, point cloud, intrinsic parameters, and algorithm output.

4.2 Synchronous Interface Call Workflow



4.3 Asynchronous Interface Call Workflow



4.4 Multi-Camera Development Workflow

Stage	Recommendation
Device planning	Assign a stable IP address to each camera. The sample creates threads by IP and opens devices independently.
Thread model	Use an independent acquisition thread for each camera. Save DcHandle, device information, image size, and data type inside the thread.
Status monitoring	Register DcRegisterCameraStatusCallback and read CameraStatus.camera_state and status_time when the status changes.
Display processing	Do not block the acquisition thread with image display or time-consuming processing. The sample uses a queue to pass display data to the display thread.

Stage	Recommendation
Bandwidth control	Enable only necessary data streams. When multiple cameras output RGBD at the same time, pay attention to network port bandwidth, CPU decoding, and OpenCV display overhead.

4.5 Built-in Application Algorithm Workflow

Step	Interface/Operation	Description
1	DcSetIntValue(LX_INT_ALGORITHM_MODE, mode)	Select MODE_AVOID_OBSTACLE, MODE_AVOID_OBSTACLE2, MODE_PALLET_LOCATE, or MODE_VISION_LOCATION.
2	DcGetStringValue(LX_STRING_ALGORITHM_VERSION)	Read the version corresponding to the current algorithm mode.
3	DcGetStringValue / DcSetStringValue(LX_STRING_ALGORITHM_PARAMS)	Read or write algorithm parameter JSON. The algorithm mode must be set first.
4	DcSpecialControl / DcSpecialControlExtend	Call control commands such as SetObstacleMode, GetObstacleIO, SetLaserData, SetOdomData, SetRelocPose, and ImportLocationMapFile according to the algorithm.
5	DcStartStream and frame acquisition	In the obstacle avoidance V2, pallet, and localization samples, LX_PTR_ALGORITHM_OUTPUT is read after LX_CMD_GET_NEW_FRAME. In callback samples, the output can be obtained from FrameInfo.app_data.
6	Parse algorithm structures	Convert to LxAvoidanceOutput, LxAvoidanceOutputN, LxPalletPose, or LxLocation according to the algorithm mode.

5. Notes on Interface Calls

5.1 Call Sequence and Resource Management

Rule	Description
Open and close in pairs	After DcOpenDevice succeeds, DcCloseDevice must be called to release resources and exclusive control permission. If the program exits abnormally without closing the device, wait for the heartbeat timeout before the permission is released.
Start and stop streaming in pairs	After DcStartStream succeeds, call DcStopStream before exit. Stop streaming before closing the device.
Unregister callbacks first	When using frame callbacks, status callbacks, or IMU callbacks, unregister the corresponding callback before closing the device.

Rule	Description
Distinguish synchronous and callback modes	In synchronous mode, LX_CMD_GET_NEW_FRAME is used to actively refresh data. In callback mode, data is pushed by the SDK, and the callback does not need to actively refresh frames.
Do not release internal memory	The internal memory returned by DcGetDeviceList, DcGetStringValue, and DcGetPtrValue usually does not need to be released externally. The content may be refreshed by the next call. Copy it yourself when using it across threads.

5.2 Preconditions for Some Interface Calls

Category	Restriction or Precondition
Streaming status restrictions	When streaming is started or LX_INT_WORK_MODE is WORK_FOREVER, parameters that must be determined before streaming cannot be set. These include ROI, 3D/2D Binning, multi-exposure HDR, multi-camera mode, trigger mode, calculate up, synchronized frame, and saving parameter groups.
ROI setting	The current interface name is DcSetROI. If offsetx, offsey, width, and height are not multiples of 8, the SDK automatically adjusts them to nearby multiples of 8. Re-read image width and height after setting ROI.
3D auto exposure enabled	When LX_BOOL_ENABLE_3D_AUTO_EXPOSURE is enabled, multi-exposure HDR, the first/second/third/fourth exposure, and gain cannot be set manually.
3D auto exposure disabled	After 3D auto exposure is disabled, LX_INT_3D_AUTO_EXPOSURE_LEVEL, LX_INT_3D_AUTO_EXPOSURE_MAX, and LX_INT_3D_AUTO_EXPOSURE_MIN are no longer used as auto exposure control items.
2D auto exposure enabled	When LX_BOOL_ENABLE_2D_AUTO_EXPOSURE is enabled, LX_INT_2D_MANUAL_EXPOSURE and LX_INT_2D_MANUAL_GAIN should not be set.
2D auto exposure disabled	After 2D auto exposure is disabled, LX_INT_2D_AUTO_EXPOSURE_LEVEL is no longer used as an auto exposure control item.
Filter mode	When LX_INT_FILTER_MODE is FILTER_NORMAL, use LX_INT_FILTER_SMOOTH_LEVEL, LX_INT_FILTER_NOISE_LEVEL, LX_INT_FILTER_TIME_LEVEL, and LX_INT_FILTER_FILL_LEVEL. FILTER_SIMPLE uses LX_FLOAT_FILTER_LEVEL. FILTER_EXPERT uses string parameters.
RGBD alignment	It is recommended to use LX_INT_RGBD_ALIGN_MODE to configure RGBD_ALIGN_OFF, DEPTH_TO_RGB, RGB_TO_DEPTH, or DEPTH_TO_RGB_SPARSE. The historical Boolean alignment item is kept only for compatibility.
Point cloud reading	To read LX_PTR_XYZ_DATA or LX_PTR_XYZIRT_DATA, the depth stream must be enabled and a valid frame must already exist. In the LiDAR

Category	Restriction or Precondition
	point cloud, timestamps and row/column indexes are affected by 3D undistortion or RGBD alignment.
Algorithm parameters	LX_INT_ALGORITHM_MODE must be set before reading or writing LX_STRING_ALGORITHM_PARAMS or LX_STRING_ALGORITHM_VERSION.
Global configuration	Global or decoding configurations such as DcSetGpuEnable, DcSetPtpEnable, DcSetParallelThread, and DcSetJpegDecodeMethod are recommended to be set before opening the device or before starting streaming.

5.3 Return Value Handling

All interfaces that return LX_STATE should have their return values checked. LX_SUCCESS indicates that the call succeeded. When an error code is returned, DcGetErrorString can be called to obtain the status description. For synchronous streaming, FrameInfo.frame_state must also be checked. Do not judge success only by whether DcSetCmd succeeded.

6. Data Types and Enumerations

6.1 LX_STATE

Status codes used by SDK interface functions and frame status.

Keyword	Value	Description
LX_SUCCESS	0	Success.
LX_ERROR	-1	Unknown error.
LX_E_NOT_SUPPORT	-2	Function not supported.
LX_E_NETWORK_ERROR	-3	Network communication error.
LX_E_INPUT_ILLEGAL	-4	Invalid input parameter.
LX_E_RECONNECTING	-5	Device reconnecting.
LX_E_DEVICE_ERROR	-6	Device fault or device response failure.
LX_E_DEVICE_NEED_UPDATE	-7	Device version is too low and must be upgraded.
LX_E_API_NEED_UPDATE	-8	API version is too low.
LX_E_CTRL_PERMISS_ERROR	-9	Exclusive control permission failed.
LX_E_GET_DEVICEINFO_ERROR	-10	Failed to obtain device information.
LX_E_IMAGE_SIZE_ERROR	-11	Image size mismatch. Reopen the device.

Keyword	Value	Description
LX_E_IMAGE_PARTITION_ERROR	-12	Image parsing failed because pixformat is incorrect. Try reopening the device.
LX_E_DEVICE_NOT_CONNECTED	-13	Camera is not connected.
LX_E_DEVICE_INIT_FAILED	-14	Camera initialization failed.
LX_E_DEVICE_NOT_FOUND	-15	Camera not found, or matching camera not found.
LX_E_FILE_INVALID	-16	File error. The file name, type, or format is incorrect, or the file failed to open.
LX_E_CRC_CHECK_FAILED	-17	File CRC or MD5 check failed.
LX_E_TIME_OUT	-18	Timeout.
LX_E_FRAME_LOSS	-19	Frame loss.
LX_E_ENABLE_ANYSTREAM_FAILED	-20	Failed to enable any stream.
LX_E_NOT_RECEIVE_STREAM	-21	No stream data received.
LX_E_PARSE_STREAM_FAILED	-22	Streaming started successfully but stream data parsing failed.
LX_E_PROCESS_IMAGE_FAILED	-23	Image computation or processing failed.
LX_E_SETTING_NOT_ALLOWED	-24	Setting is not allowed in always-on mode.
LX_E_LOAD_DATAPROCESSLIB_ERROR	-25	Error loading image processing algorithm library.
LX_E_FUNCTION_CALL_LOGIC_ERROR	-26	Function call logic error.
LX_E_IPAPPDR_UNREACHABLE_ERROR	-27	IP unreachable or network configuration error.
LX_E_FRAME_ID_NOT_MATCH	-28	Frame synchronization error within the timeout range.
LX_E_FRAME_MULTI_MACHINE	-29	Multi-camera interference signal detected in the frame.
LX_E_FRAME_IMAGE_ERROR	-30	Camera-device image-related exception detected in frame data.
LX_W_NOT_SUPPORT	2	Function not supported.
LX_W_LOAD_DATAPROCESSLIB_ERROR	25	Failed to load image processing algorithm library. Other functions are not affected.

6.2 LX_DATA_TYPE

Data formats for images, point clouds, and algorithm outputs.

Keyword	Value	Description
LX_DATA_UNSIGNED_CHAR	0	Unsigned char data
LX_DATA_CHAR	1	Signed char data
LX_DATA_UNSIGNED_SHORT	2	Unsigned short data
LX_DATA_SIGNED_SHORT	3	Signed short data
LX_DATA_INT	4	Integer data
LX_DATA_FLOAT	5	Floating-point data
LX_DATA_DOUBLE	6	Double-precision floating-point data
LX_DATA_OBSTACLE	16	Obstacle avoidance algorithm structure type
LX_DATA_PALLET	17	Pallet docking algorithm structure type
LX_DATA_LOCATION	18	Visual localization algorithm structure type
LX_DATA_OBSTACLE2	19	Obstacle avoidance algorithm V2 structure type
LX_DATA_CUSTOM	100	Camera application custom data type

6.3 LX_DEVICE_TYPE

Device model enumeration. Actual supported capabilities depend on the specific device and firmware.

Keyword	Value	Description
LX_DEVICE_M2	1001	M2 series camera
LX_DEVICE_M3	1002	M3 series camera
LX_DEVICE_M4Pro	1003	M4 Pro series camera
LX_DEVICE_M4_MEGA	1004	M4 MEGA series camera
LX_DEVICE_M4	1005	M4 series camera
LX_DEVICE_M4V1_1	1006	M4 V1.1 series camera
LX_DEVICE_M4ProV1_1	1007	M4 Pro V1.1 series camera
LX_DEVICE_S1	2001	S1 series camera
LX_DEVICE_S2	2002	S2 series camera
LX_DEVICE_S2MaxV1	2003	S2 Max V1 series camera
LX_DEVICE_S2MaxV2	2004	S2 Max V2 series camera
LX_DEVICE_S2MaxV1_1	2005	S2 Max V1.1 series camera
LX_DEVICE_S3	2101	S3 series camera

Keyword	Value	Description
LX_DEVICE_S10	2108	S10 series camera
LX_DEVICE_S10PRO	2109	S10 PRO series camera
LX_DEVICE_S10ULTRA	2110	S10 ULTRA series camera
LX_DEVICE_S11	2201	S11 series camera
LX_DEVICE_I1	3001	I1 series camera
LX_DEVICE_I2	3002	I2 series camera
LX_DEVICE_T1	4001	T1 series camera
LX_DEVICE_T2	4002	T2 series camera
LX_DEVICE_H3	5001	H3 series camera
LX_DEVICE_H4	5002	H4 series camera
LX_DEVICE_V1Pro	6001	V1 Pro series camera
LX_DEVICE_V2Pro	6002	V2 Pro series camera
LX_DEVICE_NULL	0	Reserved or invalid device type

6.4 LX_DEVICE_SERIALS

Device category search filter enumeration.

Keyword	Value	Description
LX_SERIAL_GIGE	1	GigE network camera device category
LX_SERIAL_WK	2	WK device category
LX_SERIAL_OTHER	3	Other device category
LX_SERIAL_ALL	4	Search all device categories
LX_SERIAL_LOCAL_LOOP	5	Local loopback, used to distinguish the case where the SDK and the camera-side program run on the same host

6.5 LX_OPEN_MODE

The meaning of the param parameter when opening a device depends on the open mode.

Keyword	Value	Description
OPEN_BY_INDEX	0	Open by index in the search list. The corresponding parameter is the index number. When the discovered device list changes, the selected device may also change.
OPEN_BY_IP	1	Open by the corresponding IP in the search list. The corresponding parameter is device IP or ip:port.

Keyword	Value	Description
OPEN_BY_SN	2	Open by the corresponding SN in the search list. The corresponding parameter is device SN.
OPEN_BY_ID	3	Open by the corresponding ID in the search list. The corresponding parameter is device ID.

6.6 LX_2D_OUTPUT_FORMAT

2D image output format.

Keyword	Value	Description
LX_2D_FORMAT_RAW	0	RAW original 2D image format
LX_2D_FORMAT_JPEG	4	In this mode, subsequent algorithm processing is not performed, including undistortion and alignment.
LX_2D_FORMAT_YUV_NV12	5	YUV420
LX_2D_FORMAT_YUV_UYVY	6	YUV422
LX_2D_FORMAT_RGB	8	RGB888

6.7 JPEG_DECODE_METHOD

2D JPEG image decoding method.

Keyword	Value	Description
JPEG_DECODE_TURBOJPEG	0	Turbo jpeg
JPEG_DECODE_DP	1	Use LxDataProcess
JPEG_DECODE_RKMPP	2	RK hardware decoding, dynamically loading librockchip_mpp.so
JPEG_DECODE_GPU	3	Not implemented yet

6.8 LX_BINNING_MODE

Image pixel binning mode.

Keyword	Value	Description
LX_BINNING_1X1	0	1x1 Binning, no pixel binning
LX_BINNING_2X2	1	2x2 pixel binning
LX_BINNING_4X4	2	4x4 pixel binning

6.9 LX_STRUCT_LIGHT_CODE_MODE

Structured-light camera coding mode.

Keyword	Value	Description
LX_CODE_NORMAL	1	Normal
LX_CODE_STATBLE	2	Stable
LX_CODE_ENHANCE	3	Enhanced high precision

6.10 LX_ALGORITHM_MODE

Camera built-in application algorithm mode.

Keyword	Value	Description
MODE_ALL_OFF	0	Disable built-in obstacle avoidance algorithm
MODE_AVOID_OBSTACLE	1	Built-in obstacle avoidance algorithm
MODE_PALLET_LOCATE	2	Built-in pallet docking algorithm
MODE_VISION_LOCATION	3	Built-in visual localization algorithm
MODE_AVOID_OBSTACLE2	4	Built-in obstacle avoidance algorithm V2
MODE_GENERIC_DATA	5	Custom data type, application algorithm platform

6.11 LX_CAMERA_WORK_MODE

Camera work mode.

Keyword	Value	Description
KEEP_HEARTBEAT	0	Keep heartbeat. The camera automatically enters standby after the SDK heartbeat is interrupted.
WORK_FOREVER	1	Always keep working. Some parameters cannot be set in this mode.

6.12 LX_TRIGGER_MODE

Trigger mode.

Keyword	Value	Description
LX_TRIGGER_MODE_OFF	0	Trigger mode off. Stream mode. Default.
LX_TRIGGER_SOFTWARE	1	Software trigger mode
LX_TRIGGER_HARDWARE	2	Hardware trigger mode

6.13 LX_IO_WORK_MODE

IO work mode.

Keyword	Value	Description
ALGORITHM_IO_MODE	0	IO is used as input/output for the built-in application algorithm.
USER_IO_MODE	1	The user can control the IO status. Refer to LX_IO_OUT_USERCTRL_MODE.
TRIGGER_IO_MODE	2	IO is used as trigger signal output. Valid only in trigger mode.

6.14 LX_IO_OUTPUT_STATE

User-controlled IO output combination.

Keyword	Value	Description
OUT1_0_OUT2_0	0	OUT1 outputs 0, OUT2 outputs 0
OUT1_0_OUT2_1	1	OUT1 outputs 0, OUT2 outputs 1
OUT1_1_OUT2_0	2	OUT1 outputs 1, OUT2 outputs 0
OUT1_1_OUT2_1	3	OUT1 outputs 1, OUT2 outputs 1

6.15 LX_IO_HARDWARE_MODE

IO output hardware work mode.

Keyword	Value	Description
OPEN_DRAIN_MODE	0	Open-drain normally-open mode
PUSH_PULL_MODE	1	Push-pull high-level mode

6.16 LX_FILTER_MODE

Filter parameter mode.

Keyword	Value	Description
FILTER_SIMPLE	1	Only one parameter: LX_FLOAT_FILTER_LEVEL
FILTER_NORMAL	2	Divided into multiple modules: smoothing, denoising, filling, and temporal filtering
FILTER_EXPERT	3	Configure detailed algorithms through strings

6.17 LX_3D_FREQ_MODE

3D frequency mode.

Keyword	Value	Description
FREQ_SINGLE_3D	0	Single frequency

Keyword	Value	Description
FREQ_MULTI_3D	1	Multi-frequency
FREQ_TRIPLE_3D	2	Triple frequency

6.18 LX_RGBD_ALIGN_MODE

RGBD alignment mode.

Keyword	Value	Description
RGBD_ALIGN_OFF	0	Disable RGBD alignment. RGB and depth use their own resolutions and coordinate systems.
DEPTH_TO_RGB	1	Align depth to RGB. Resolution and coordinates are based on RGB, and RGB is forced to be undistorted.
RGB_TO_DEPTH	2	Align RGB to depth. Resolution and coordinates are based on depth.
DEPTH_TO_RGB_SPARSE	3	Computing-optimized DEPTH_TO_RGB. Resolution and coordinates are based on RGB, but the valid point cloud data amount is not changed.

6.19 LX_CAN_PROTOCOL_TYPE

CAN protocol type.

Keyword	Value	Description
CAN_MRDVS	0	MRDVS CAN protocol
CAN_KECONG	1	Kecong CAN protocol

6.20 LX_POWER_MODE_TYPE

Camera power mode.

Keyword	Value	Description
POWER_MODE_LOW	0	Low-power mode, with some functions disabled
POWER_MODE_DEFAULT	1	Default power mode, balancing performance and power consumption
POWER_MODE_HIGH	2	High-performance mode, enabling all functions
POWER_MODE_STANDBY	3	Standby mode, with the lowest power consumption and image acquisition suspended

6.21 LX_CAMERA_STATUS

Status enumeration in the camera status callback.

Keyword	Value	Description
STATUS_CLOSED	0	Camera is not opened or is closed
STATUS_OPENED_UNSTARTED	1	Camera is opened but streaming has not started
STATUS_STARTED	2	Camera is opened and streaming has started
STATUS_CONNECTING	3	Camera is disconnected and reconnecting
STATUS_CONNECT_SUCCESS	4	Camera reconnected successfully

6.22 LX_DISTORTION_MODEL

Distortion model type in the camera intrinsic parameter structure.

Keyword	Value	Description
LX_DISTORTION_UNDEFINE	0	Invalid distortion model
LX_DISTORTION_RADTAN_5	5	Normal radial-tangential model [k1, k2, p1, p2, k3]
LX_DISTORTION_RADTAN_14	14	Large-distortion radial-tangential model [k1, k2, p1, p2, k3, k4, k5, k6, s1, s2, s3, s4, t1, t2]
LX_DISTORTION_FISHEYE	15	Fisheye distortion model [k1, k2, k3, k4]
LX_DISTORTION_SCARAMUZZA	16	Scaramuzza distortion model

7. Feature Parameter Description

7.1 Int Parameters

Parameter	Number	Access Interface	Description
LX_INT_FIRST_EXPOSURE	1001	DcSetIntValue / DcGetIntValue	Default high-integration exposure value, in us. For multi-integration cases, this is the exposure time of the first integration.
LX_INT_SECOND_EXPOSURE	1002	DcSetIntValue / DcGetIntValue	Default low-integration exposure value, in us. For multi-integration cases, this is the exposure time of the second integration.
LX_INT_THIRD_EXPOSURE	1003	DcSetIntValue / DcGetIntValue	For multi-integration cases, this is the exposure time of the third integration, in us.
LX_INT_FOURTH_EXPOSURE	1004	DcSetIntValue / DcGetIntValue	For multi-integration cases, this is the exposure time of the fourth integration, in us.

Parameter	Number	Access Interface	Description
LX_INT_GAIN	1005	DcSetIntValue / DcGetIntValue	Gain, equivalent to the exposure effect. It introduces noise. Adjust the gain properly to avoid excessive exposure parameters.
LX_INT_MIN_DEPTH	1011	DcSetIntValue / DcGetIntValue	Minimum depth value
LX_INT_MAX_DEPTH	1012	DcSetIntValue / DcGetIntValue	Maximum depth value
LX_INT_MIN_AMPLITUDE	1013	DcSetIntValue / DcGetIntValue	Minimum amplitude value of valid signals
LX_INT_MAX_AMPLITUDE	1014	DcSetIntValue / DcGetIntValue	Maximum amplitude value of valid signals
LX_INT_CODE_MODE	1016	DcSetIntValue / DcGetIntValue	Structured-light camera coding mode. Refer to LX_STRUCT_LIGHT_CODE_MODE.
LX_INT_WORK_MODE	1018	DcSetIntValue / DcGetIntValue	Work mode. Refer to LX_CAMERA_WORK_MODE.
LX_INT_LINK_SPEED	1019	DcGetIntValue	Negotiated network adapter speed. 100 means 100 Mbps and 1000 means gigabit. Read-only.
LX_INT_3D_IMAGE_WIDTH	1021	DcGetIntValue	3D image resolution width. It changes after ROI/Binning/2D alignment and must be obtained again.
LX_INT_3D_IMAGE_HEIGHT	1022	DcGetIntValue	3D image resolution height. It changes after ROI/Binning/2D alignment and must be obtained again.
LX_INT_3D_IMAGE_OFFSET_X	1023	DcSetIntValue / DcGetIntValue	ROI horizontal pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters.
LX_INT_3D_IMAGE_OFFSET_Y	1024	DcSetIntValue / DcGetIntValue	ROI vertical pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters.
LX_INT_3D_BINNING_MODE	1025	DcSetIntValue / DcGetIntValue	3D image binning. Refer to LX_BINNING_MODE.

Parameter	Number	Access Interface	Description
LX_INT_3D_DEPTH_DATA_TYPE	1026	DcGetIntValue	Depth image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value.
LX_INT_3D_FREQ_MODE	1027	DcSetIntValue / DcGetIntValue	3D frequency mode. See the definition of LX_3D_FREQ_MODE.
LX_INT_3D_AMPLITUDE_CHANNEL	1031	DcSetIntValue / DcGetIntValue	Number of 3D amplitude image channels. It shares channels with the depth image. Monochrome is 1 and color is 3.
LX_INT_3D_AMPLITUDE_DATA_TYPE	1035	DcGetIntValue	Amplitude image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value.
LX_INT_3D_AUTO_EXPOSURE_LEVEL	1036	DcSetIntValue / DcGetIntValue	Exposure level when 3D auto exposure is enabled. Exposure value and gain cannot be set during this period.
LX_INT_3D_AUTO_EXPOSURE_MAX	1037	DcSetIntValue / DcGetIntValue	Upper limit of 3D auto exposure
LX_INT_3D_AUTO_EXPOSURE_MIN	1038	DcSetIntValue / DcGetIntValue	Lower limit of 3D auto exposure
LX_INT_2D_IMAGE_WIDTH	1041	DcSetIntValue / DcGetIntValue	2D image resolution width. It changes after ROI or Binning.
LX_INT_2D_IMAGE_HEIGHT	1042	DcSetIntValue / DcGetIntValue	2D image resolution height. It changes after ROI or Binning.
LX_INT_2D_IMAGE_OFFSET_X	1043	DcSetIntValue / DcGetIntValue	2D image ROI horizontal pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters.
LX_INT_2D_IMAGE_OFFSET_Y	1044	DcSetIntValue / DcGetIntValue	2D image ROI vertical pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters.
LX_INT_2D_BINNING_MODE	1045	DcSetIntValue / DcGetIntValue	2D image binning. Refer to LX_BINNING_MODE.
LX_INT_2D_IMAGE_CHANNEL	1046	DcSetIntValue / DcGetIntValue	Number of 2D image channels. Monochrome and JPEG are 1, color is 3, and YUV422 is 2.

Parameter	Number	Access Interface	Description
LX_INT_2D_IMAGE_DATA_TYPE	1047	DcGetIntValue	2D image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value.
LX_INT_2D_IMAGE_OUTPUT_FORMAT	1048	DcSetIntValue / DcGetIntValue	2D image output format. Refer to LX_2D_OUTPUT_FORMAT.
LX_INT_2D_ENCODE_RATIO	1049	DcSetIntValue / DcGetIntValue	By default, 2D images are compressed as JPEG. This parameter sets the compression ratio.
LX_INT_2D_MANUAL_EXPOSURE	1051	DcSetIntValue / DcGetIntValue	Exposure value for 2D manual exposure
LX_INT_2D_MANUAL_GAIN	1052	DcSetIntValue / DcGetIntValue	Gain value for 2D manual exposure
LX_INT_2D_AUTO_EXPOSURE_LEVEL	1054	DcSetIntValue / DcGetIntValue	Exposure level for 2D image auto exposure [0-100]. 0 means darker overall, and 100 means brighter overall.
LX_INT_TOF_GLOBAL_OFFSET	1061	DcSetIntValue / DcGetIntValue	3D depth data offset
LX_INT_3D_UNDISTORT_SCALE	1062	DcSetIntValue / DcGetIntValue	3D image undistortion coefficient [0,100]. The larger the value, the more field of view is retained. 0 keeps the original intrinsic parameters, and 1 means no black border.
LX_INT_ALGORITHM_MODE	1065	DcSetIntValue / DcGetIntValue	Set the built-in application algorithm. Supported by some models. Refer to LX_ALGORITHM_MODE for the corresponding value.
LX_INT_MODBUS_ADDRESS	1066	DcSetIntValue / DcGetIntValue	Modbus address. Some models support MODBUS protocol output through serial port.
LX_INT_HEART_TIME	1067	DcSetIntValue / DcGetIntValue	Heartbeat time with the device, in ms
LX_INT_GVSP_PACKET_SIZE	1068	DcSetIntValue / DcGetIntValue	GVSP single-packet data segment size, in bytes
LX_INT_TRIGGER_MODE	1069	DcSetIntValue / DcGetIntValue	Trigger mode. Refer to LX_TRIGGER_MODE for the corresponding value.
LX_INT_CALCULATE_UP	1070	DcSetIntValue / DcGetIntValue	Allows 2D or 3D algorithm computation to be migrated

Parameter	Number	Access Interface	Description
			between the host and the camera to save computing power on either side. This may affect frame rate and latency.
LX_INT_CAN_BAUD_RATE	1072	DcSetIntValue / DcGetIntValue	CAN baud rate value, in bps
LX_INT_CAN_NODE_ID	1073	DcSetIntValue / DcGetIntValue	CAN address
LX_INT_CAN_PROTOCOL_TYPE	1074	DcSetIntValue / DcGetIntValue	CAN protocol type. Refer to LX_CAN_PROTOCOL_TYPE.
LX_INT_SAVE_PARAMETERS_GROUP	1075	DcSetIntValue / DcGetIntValue	Save the current camera configuration as the specified parameter group.
LX_INT_LOAD_PARAMETERS_GROUP	1076	DcSetIntValue / DcGetIntValue	Load a parameter group with the specified index in one step.
LX_INT_RGBD_ALIGN_MODE	1077	DcSetIntValue / DcGetIntValue	RGBD alignment mode. Refer to LX_RGBD_ALIGN_MODE. This replaces the historical compatibility item LX_BOOL_ENABLE_2D_TO_DEPTH.
LX_INT_XYZ_COORDINATE	1078	DcSetIntValue / DcGetIntValue	Point cloud coordinate system. 0: camera coordinate system, where xyz is right-down-forward. 1: robot coordinate system, where xyz is forward-left-up.
LX_INT_XYZ_UNIT	1079	DcSetIntValue / DcGetIntValue	Point cloud unit. 0: mm, 1: m. This only affects the data of LX_PTR_XYZ_DATA and LX_PTR_XYZIRT_DATA.
LX_INT_GVCP_TIMEOUT	1080	DcGetIntValue	Maximum timeout for device control and information acquisition. Default is 1800ms, in ms.
LX_INT_HARDWARE_TRIGGER_FILTER_TIME	1085	DcSetIntValue / DcGetIntValue	Hardware trigger filter time, in us
LX_INT_TRIGGER_MIN_PERIOD_TIME	1086	DcSetIntValue / DcGetIntValue	Minimum trigger interval, in us
LX_INT_TRIGGER_DELAY_TIME	1087	DcSetIntValue / DcGetIntValue	Trigger delay time, in us
LX_INT_TRIGGER_FRAME_COUNT	1088	DcSetIntValue / DcGetIntValue	Number of frames per trigger

Parameter	Number	Access Interface	Description
LX_INT_FILTER_MODE	1090	DcSetIntValue / DcGetIntValue	Filter mode. Refer to LX_FILTER_MODE.
LX_INT_FILTER_SMOOTH_LEVEL	1091	DcSetIntValue / DcGetIntValue	When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter smoothing level [0, 3]. The larger the value, the stronger the filtering.
LX_INT_FILTER_NOISE_LEVEL	1092	DcSetIntValue / DcGetIntValue	When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter noise level [0, 3]. The larger the value, the stronger the filtering.
LX_INT_FILTER_TIME_LEVEL	1093	DcSetIntValue / DcGetIntValue	When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the temporal filter level [0, 3]. The larger the value, the stronger the filtering.
LX_INT_FILTER_DETECT_LOW_SIGNAL	1094	DcSetIntValue / DcGetIntValue	Low-signal measurement. It may detect data with a low signal-to-noise ratio, including long-distance data that causes multi-period problems.
LX_INT_FILTER_FILL_LEVEL	1095	DcSetIntValue / DcGetIntValue	When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter filling level [0, 3]. The larger the value, the stronger the filtering.
LX_INT_IMU_ACCELERATION_LEVEL	1101	DcSetIntValue / DcGetIntValue	IMU acceleration range level [0-3]. The larger the value, the larger the range. Each level doubles the range.
LX_INT_IMU_ANGULAR_RANGE_LEVEL	1102	DcSetIntValue / DcGetIntValue	IMU angular velocity range level [0-4]. The larger the value, the larger the range. Each level doubles the range.
LX_INT_MAX_TOLERANT_LOST_PACKETS	1103	DcSetIntValue / DcGetIntValue	Number of lost packets allowed in one frame. If the threshold is exceeded, the frame is discarded. Otherwise, lost packets are filled with 0. Default is 5.
LX_INT_3D_FPS	1510	DcSetIntValue / DcGetIntValue	Frame rate of depth and amplitude
LX_INT_2D_UNDISTORT_SCALE	1520	DcSetIntValue / DcGetIntValue	2D image undistortion coefficient [0,100]. The larger the value, the more field of view is retained. 0 keeps the original intrinsic

Parameter	Number	Access Interface	Description
			parameters, and 1 means no black border.
LX_INT_IO_WORK_MODE	1530	DcSetIntValue / DcGetIntValue	GPIO signal output control mode. Refer to LX_IO_WORK_MODE.
LX_INT_IO_OUTPUT_STATE	1531	DcSetIntValue / DcGetIntValue	User control mode of GPIO signal output. Refer to LX_IO_OUTPUT_STATE.
LX_INT_IO_HARDWARE_WORK_MODE	1532	DcSetIntValue / DcGetIntValue	Hardware work mode of GPIO signal output. Refer to LX_IO_HARDWARE_MODE.
LX_INT_IO_INPUT_STATUS	1533	DcGetIntValue	IO input status. Read-only.
LX_INT_IO_OUTPUT_STATUS	1534	DcGetIntValue	IO output status. Read-only.
LX_INT_3D_GLARE_LEVEL	1738	DcSetIntValue / DcGetIntValue	Glare level. 0 means off. Levels are 1, 2, and 3.
LX_INT_POWER_MODE	1755	DcSetIntValue / DcGetIntValue	Camera power mode. See the definition of LX_POWER_MODE_TYPE. Supported by some models.

7.2 Float Parameters

Parameter	Number	Access Interface	Description
LX_FLOAT_FILTER_LEVEL	2001	DcSetFloatValue / DcGetFloatValue	When LX_INT_FILTER_MODE is FILTER_SIMPLE, this sets the filter level [0, 1]. The larger the value, the stronger the filtering. A value of 0 means filtering is disabled.
LX_FLOAT_EST_OUT_EXPOSURE	2002	DcSetFloatValue / DcGetFloatValue	Whether to evaluate overexposed data [0, 1]. If set to 1, overexposed data is invalid.
LX_FLOAT_LIGHT_INTENSITY	2003	DcSetFloatValue / DcGetFloatValue	Light intensity [0, 1]. Supported by some models.
LX_FLOAT_3D_DEPTH_FPS	2004	DcGetFloatValue	Current frame rate of the depth image. Read-only.
LX_FLOAT_3D_AMPLITUDE_FPS	2005	DcGetFloatValue	Current frame rate of the amplitude image. Read-only.
LX_FLOAT_2D_IMAGE_FPS	2006	DcGetFloatValue	Current frame rate of the 2D/RGB image. Read-only.

Parameter	Number	Access Interface	Description
LX_FLOAT_DEVICE_TEMPERATURE	2007	DcGetFloatValue	Current camera temperature. Read-only.
LX_FLOAT_CONFIDENCE	2008	DcSetFloatValue / DcGetFloatValue	Confidence [0-1]. The smaller the value, the more data is retained.

7.3 Bool Parameters

Parameter	Number	Access Interface	Description
LX_BOOL_CONNECT_STATE	3001	DcSetBoolValue / DcGetBoolValue	Current connection status
LX_BOOL_ENABLE_3D_DEPTH_STREAM	3002	DcSetBoolValue / DcGetBoolValue	Enable/disable depth data stream, supported by some cameras
LX_BOOL_ENABLE_3D_AMP_STREAM	3003	DcSetBoolValue / DcGetBoolValue	Enable/disable amplitude data stream, supported by some cameras
LX_BOOL_ENABLE_3D_AUTO_EXPOSURE	3006	DcSetBoolValue / DcGetBoolValue	Enable 3D auto exposure
LX_BOOL_ENABLE_3D_UNDISTORT	3007	DcSetBoolValue / DcGetBoolValue	Enable 3D undistortion
LX_BOOL_ENABLE_ANTI_FLICKER	3008	DcSetBoolValue / DcGetBoolValue	Enable anti-flicker. LED ambient lighting may cause obvious ripples in data. Supported by some models.
LX_BOOL_ENABLE_2D_STREAM	3011	DcSetBoolValue / DcGetBoolValue	Enable/disable 2D data stream
LX_BOOL_ENABLE_2D_AUTO_EXPOSURE	3012	DcSetBoolValue / DcGetBoolValue	Enable 2D auto exposure
LX_BOOL_ENABLE_2D_UNDISTORT	3015	DcSetBoolValue / DcGetBoolValue	Enable 2D image undistortion
LX_BOOL_ENABLE_2D_TO_DEPTH	3016	DcSetBoolValue / DcGetBoolValue	LX_INT_RGBD_ALIGN_MODE is recommended.
LX_BOOL_ENABLE_BACKGROUND_AMP	3017	DcSetBoolValue / DcGetBoolValue	Enable amplitude background light
LX_BOOL_ENABLE_MULTI_MACHINE	3018	DcSetBoolValue / DcGetBoolValue	Enable multi-camera mode. TOF may have multi-camera interference. Enabling this mode can automatically detect and mitigate multi-camera interference.
LX_BOOL_ENABLE_MULTI_EXPOSURE_HDR	3019	DcSetBoolValue / DcGetBoolValue	Enable HDR, multi-exposure high dynamic range mode

Parameter	Number	Access Interface	Description
LX_BOOL_ENABLE_SYNC_FRAME	3020	DcSetBoolValue / DcGetBoolValue	Whether to enable forced frame synchronization. By default, data real-time performance has priority. If RGBD synchronization is required, enable this mode.
LX_BOOL_ENABLE_TERM_RESISTOR	3021	DcSetBoolValue / DcGetBoolValue	Enable termination resistor
LX_BOOL_ENABLE_IMU	3023	DcSetBoolValue / DcGetBoolValue	Enable IMU data. Obtain data through the DcRegisterImuDataCallback callback.
LX_BOOL_ENABLE_LASER_BRIGHT	3110	DcSetBoolValue / DcGetBoolValue	Enable laser on/off control

7.4 String Parameters

Parameter	Number	Access Interface	Description
LX_STRING_DEVICE_VERSION	4001	DcSetStringValue / DcGetStringValue	Device version number
LX_STRING_FIRMWARE_NAME	4003	DcSetStringValue	Firmware file name, used to upgrade the device version. Some models need to reopen the camera.
LX_STRING_FILTER_PARAMS	4004	DcSetStringValue / DcGetStringValue	FILTER_NORMAL is recommended.
LX_STRING_ALGORITHM_PARAMS	4005	DcSetStringValue / DcGetStringValue	Built-in algorithm parameters. According to the currently set LX_ALGORITHM_MODE, returns the corresponding JSON format string.
LX_STRING_ALGORITHM_VERSION	4006	DcSetStringValue / DcGetStringValue	Built-in algorithm version number. According to the currently set LX_ALGORITHM_MODE, returns the corresponding version number.
LX_STRING_DEVICE_OS_VERSION	4007	DcSetStringValue / DcGetStringValue	Device system image version number
LX_STRING_IMPORT_PARAMS_FROM_FILE	4008	DcSetStringValue / DcGetStringValue	Load parameters from a local file to the camera
LX_STRING_EXPORT_PARAMS_TO_FILE	4009	DcSetStringValue / DcGetStringValue	Export current camera parameters to a local file

Parameter	Number	Access Interface	Description
LX_STRING_CUSTOM_ID	4010	DcSetStringValue / DcGetStringValue	Customer-defined device ID identifier

7.5 Command Parameters

Parameter	Number	Access Interface	Description
LX_CMD_GET_NEW_FRAME	5001	DcSetCmd	Actively update the latest current data. Call this before obtaining LX_PTR_FRAME_DATA, LX_PTR_XYZ_DATA, LX_PTR_XYZIRT_DATA, or algorithm output in synchronous streaming. It is not required in callback mode.
LX_CMD_RETURN_VERSION	5002	DcSetCmd	Roll back to the previous version
LX_CMD_RESTART_DEVICE	5003	DcSetCmd	Restart the camera. Some models need to reopen the camera.
LX_CMD_RESET_PARAM	5007	DcSetCmd	Restore default parameters
LX_CMD_SOFTWARE_TRIGGER	5008	DcSetCmd	Software trigger execution command
LX_CMD_GET_NEW_FRAME_3D	5009	DcSetCmd	Process and update only 3D data depth/amp. Use this when reducing 2D processing overhead is required.
LX_CMD_GET_NEW_FRAME_2D	5010	DcSetCmd	Process and update only 2D/RGB data. Use this when reducing 3D processing overhead is required.

7.6 Ptr Parameters

Parameter	Number	Access Interface	Description
LX_PTR_2D_IMAGE_DATA	6001	DcGetPtrValue	Compatibility interface. Not recommended for priority use in new projects. Use rgb_data in LX_PTR_FRAME_DATA.
LX_PTR_3D_AMP_DATA	6002	DcGetPtrValue	Compatibility interface. Not recommended for priority use in new projects. Use amp_data in LX_PTR_FRAME_DATA.
LX_PTR_3D_DEPTH_DATA	6003	DcGetPtrValue	Compatibility interface. Not recommended for priority use in new

Parameter	Number	Access Interface	Description
			projects. Use depth_data in LX_PTR_FRAME_DATA.
LX_PTR_XYZ_DATA	6004	DcGetPtrValue	Obtain ordinary XYZ point cloud. Float three-channel data is arranged in x, y, z order. The depth stream must be enabled and frame acquisition must be completed.
LX_PTR_2D_INTRIC_PARAM	6005	DcGetPtrValue	Compatibility interface. Use LX_PTR_2D_INTRINSIC_PARAMETERS.
LX_PTR_3D_INTRIC_PARAM	6006	DcGetPtrValue	Compatibility interface. Use LX_PTR_3D_INTRINSIC_PARAMETERS.
LX_PTR_3D_EXTRIC_PARAM	6007	DcGetPtrValue	Obtain 3D image extrinsic parameters. The pointer type is float*, and the fixed length is 12*sizeof(float). The first 9 values represent the rotation matrix and the last 3 values represent the translation vector.
LX_PTR_ALGORITHM_OUTPUT	6008	DcGetPtrValue	Compatibility interface, used to directly read the current algorithm output. The obstacle avoidance V2, pallet, and localization samples can still use it. For general frame callback scenarios, FrameInfo.app_data is preferred.
LX_PTR_FRAME_DATA	6009	DcGetPtrValue	Obtain complete one-frame data and output FrameInfo. Recommended for unified reading of depth, amplitude, RGB, and algorithm output.
LX_PTR_2D_NEW_INTRIC_PARAM	6010	DcGetPtrValue	Use LX_PTR_2D_INTRINSIC_PARAMETERS instead. The pointer type is float*, and the fixed length is 18, including 4 intrinsic parameters and 14 distortion parameters.
LX_PTR_3D_NEW_INTRIC_PARAM	6011	DcGetPtrValue	Use LX_PTR_3D_INTRINSIC_PARAMETERS instead. The pointer type is float*, and the fixed length is 18, including 4 intrinsic parameters and 14 distortion parameters.

Parameter	Number	Access Interface	Description
LX_PTR_2D_INTRINSIC_PARAMETERS	6013	DcGetPtrValue	Obtain 2D image intrinsic parameters. Refer to LxIntrinsicParameters.
LX_PTR_3D_INTRINSIC_PARAMETERS	6014	DcGetPtrValue	Obtain 3D image intrinsic parameters. Refer to LxIntrinsicParameters.
LX_PTR_IMU_EXTRIC_PARAM	6015	DcGetPtrValue	Obtain IMU image extrinsic parameters. The pointer type is float*, and the fixed length is 12*sizeof(float). The first 9 values represent the rotation matrix and the last 3 values represent the translation vector.
LX_PTR_XYZIRT_DATA	6020	DcGetPtrValue	Obtain the LiDAR point cloud structure LxPointCloudData, including timebase, point_num, and the LxPointXYZIRT point array.

8. Structure Description

8.1 Device and Parameter Information Structures

LxDeviceInfo

Field	Type	Description
handle	DCHandle	Unique device identifier. Subsequent interfaces after opening the device depend on this handle.
dev_type	LX_DEVICE_TYPE	Device type
id	char[32]	Device ID
ip	char[32]	Device IP and port
sn	char[32]	Device serial number
mac	char[32]	Device MAC address
firmware_ver	char[32]	Device software version number
algor_ver	char[32]	Device algorithm version number
name	char[32]	Device name
reserve	char[32]	Reserved field, currently used for subnet mask
reserve2	char[32]	Reserved field, currently used for gateway IP
reserve3	char[64]	Reserved field. [0:3] can indicate the current device access mode.

Field	Type	Description
reserve4	char[128]	Reserved field

LxIntValueInfo / LxFloatValueInfo

Field	Type	Description
set_available	bool	Whether the current value can be set. true means settable, and false means not settable.
cur_value	int / float	Current value
max_value	int / float	Maximum value
min_value	int / float	Minimum value
reserve	int[4] / float[4]	Reserved field

8.2 Frame Data Structures**FrameDataInfo**

Field	Type	Description
frame_data_type	LX_DATA_TYPE	Frame data type
frame_width	int	Image width
frame_height	int	Image height
frame_channel	int	Number of channels
frame_data	void*	Pointer to frame data content. Memory is managed internally.
sensor_timestamp	unsigned long long	Sensor image output timestamp
recv_timestamp	unsigned long long	Timestamp when frame data reception is completed

FrameInfo

Field	Type	Description
frame_state	LX_STATE	Current frame status. Check whether it is LX_SUCCESS first.
handle	DcHandle	Device handle
depth_data	FrameDataInfo	Depth image data
amp_data	FrameDataInfo	Amplitude image data
rgb_data	FrameDataInfo	2D/RGB image data
app_data	FrameDataInfo	Algorithm output data. The type is determined by the current algorithm mode.

Field	Type	Description
reserve_data	void*	Extended reserved field. Currently it can be converted to FrameExtendInfo to obtain frame ID.

FrameExtendInfo

Field	Description
depth_frame_id	Depth image frame ID
amp_frame_id	Amplitude image frame ID
rgb_frame_id	RGB image frame ID
app_frame_id	Application algorithm output frame ID
reserve_data1/2/3	Reserved fields

8.3 Status Callback and IMU Structures

CameraStatus

Field	Type	Description
camera_state	LX_CAMERA_STATUS	Camera status
status_time	unsigned long long	Status change time
reserve_data	void*	Extended reserved field

LxImuData

Field	Type	Description
frame_state	LX_STATE	IMU data status
handle	DcHandle	Device handle
imu_raw_data	LxImuRawData	Raw IMU data, including acc_x/y/z, gry_x/y/z, status, and sensor_time
imu_data	LxImuSinkData	IMU data after range conversion. Acceleration unit is m/s ² , and angular velocity unit is rad/s.

8.4 Point Cloud and Camera Intrinsic Structures

LxPointXYZIRT / LxPointCloudData

Field	Type	Description
x/y/z	float	Point coordinates
intensity	uint32_t	Point intensity data. Valid only after the amplitude image is enabled.
offset_time	int32_t	Time offset relative to timebase, in us

Field	Type	Description
roi_index	uint8_t	ROI region index
row_pos/col_pos	uint16_t	Row and column position of the point in the image
timebase	uint64_t	Base timestamp when exposure starts for this frame image, in us
point_num	uint32_t	Number of valid points
points	LxPointXYZIRT*	Internally managed point array

LxIntrinsicParameters

Field	Type	Description
width/height	uint32_t	Image width and height
intrinsics	float[9]	3x3 intrinsic matrix, in the order of fx, 0, cx, 0, fy, cy, 0, 0, 1
distortion_model	LX_DISTORTION_MODEL	Distortion model type
distortion_coeffs	float*	Pointer to distortion coefficients. Memory is managed internally.
num_distortion_coeffs	uint8_t	Number of distortion coefficients

8.5 Application Algorithm Structures

Obstacle Avoidance Output

Structure	Key Fields	Description
LxAvoidanceOutput	state, groundPlane, number_3d, cloud_output, number_box, obstacleBoxs	Obstacle avoidance V1 output. The output is valid when state is LxAvSuccess.
LxAvoidanceOutputN	state, groundPlane, number_3d, cloud_output, number_box, obstacleBoxs, reservedData	Obstacle avoidance V2 output. The obstacle Box uses LxObstacleBoxN, which includes semantic type, 2D box ratio, ID, and other fields.
LxObstacleBox	pose, width, height, depth, center, type, id, velocity	Obstacle 3D Box. Some semantic and velocity fields are reserved or not implemented yet.
LxObstacleBoxN	pose, width, height, depth, center, type_idx, id, prev_id, box_2d_*, type_name	Obstacle avoidance V2 obstacle 3D/2D and semantic information

Pallet and Localization Output

Structure	Field	Description
LxPalletPose	return_val, x, y, yaw, extents	Pallet center position and tilt angle. x is the distance from the pallet center to the camera optical center, y is the rightward offset, and yaw is the horizontal tilt angle.
LxRelocPose	x, y, theta	Relocalization pose
LxOdomData	timestamp, x, y, theta	Odometry data
LxLaser	timestamp, angle_min/max, angle_increment, range_min/max, range_size, ranges	Laser data input structure. ranges is allocated by the user and released after use.
LxLocation	timestamp, status, x, y, theta, extents	Visual localization output result

Common Indexes of LxPalletPose.extents

Index	Meaning
0	Pallet height
1	Pallet length, that is, the distance from the center of the left leg to the center of the right leg
2	Number of pallet legs
3-5	Width of each pallet leg
7-8	Width of the left and right pallet holes
10	Total number of pallets n, with a maximum valid quantity of 4
11-26	x, y, theta*1000, and height of p1-p4
27-30	Left leg x/y and right leg x/y
32	Whether goods are present, 0 or 1
33-36	Result image width, height, channel, and image pointer information

9. API Description

9.1 Version, Logs, and Global Configuration

9.1.1 DcGetApiVersion

Item	Description
Function prototype	const char* DcGetApiVersion();
Function purpose	Obtain the API version number string.
Function parameters	No input parameters.

Item	Description
Return value and remarks	Returns an internally managed const char* string.

9.1.2 DcSetInfoOutput

Item	Description
Function prototype	LX_STATE DcSetInfoOutput(int print_level = 1, bool enable_screen_print = false, const char* log_path = "", int language = 0);
Function purpose	Set SDK log output level, screen printing, and log path.
Function parameters	print_level: 0 info, 1 warn, 2 error, 3 no output; enable_screen_print: whether to output to the window; log_path: log save path; language: language parameter.
Return value and remarks	Returns LX_SUCCESS if the setting succeeds. It is recommended to call this during program startup. The pure C interface is a 3-parameter version. Samples commonly use DcSetInfoOutput(1,true,"") or DcSetInfoOutput(1,false,"/").

9.1.3 DcLog

Item	Description
Function prototype	LX_STATE DcLog(const char* str);
Function purpose	Allows users to write custom debug information to SDK logs.
Function parameters	str: string ending with \0.
Return value and remarks	Returns LX_SUCCESS if the write request succeeds.

9.1.4 DcGetErrorString

Item	Description
Function prototype	const char* DcGetErrorString(LX_STATE state);
Function purpose	Obtain the status description corresponding to LX_STATE.
Function parameters	state: status code returned by the interface.
Return value and remarks	Used for unified error log output, making it easier to troubleshoot timeout, network, and parameter errors.

9.1.5 DcSetGpuEnable / DcGetGpuEnable

Item	Description
Function prototype	LX_STATE DcSetGpuEnable(bool is_enable); LX_STATE DcGetGpuEnable(bool* is_enable);
Function purpose	Set or obtain the GPU acceleration switch of the algorithm library.

Item	Description
Function parameters	is_enable: whether to enable GPU acceleration.
Return value and remarks	It is valid only when called before opening the camera. Some GPU devices may not be supported. If an exception occurs, disabling it is recommended.

9.1.6 DcSetPtpEnable / DcGetPtpEnable

Item	Description
Function prototype	LX_STATE DcSetPtpEnable(bool is_enable); LX_STATE DcGetPtpEnable(bool* is_enable);
Function purpose	Set or obtain the PTP time synchronization source service switch.
Function parameters	is_enable: whether to enable.
Return value and remarks	It is valid only when called before opening the camera. It is enabled by default.

9.1.7 DcSetParallelThread

Item	Description
Function prototype	LX_STATE DcSetParallelThread(int thread_num);
Function purpose	Set the number of parallel computing threads.
Function parameters	thread_num: number of threads. The default value -1 means automatic adjustment.
Return value and remarks	Enabling parallel computing increases CPU consumption but can improve computation speed.

9.1.8 DcSetJpegDecodeMethod

Item	Description
Function prototype	LX_STATE DcSetJpegDecodeMethod(int mode);
Function purpose	Set the 2D JPEG image decoding method.
Function parameters	mode: refer to JPEG_DECODE_METHOD.
Return value and remarks	It is recommended to set this before starting streaming. Depending on the platform, turbojpeg, LxDataProcess, or RKMPP can be selected.

9.1.9 DcSetMaxLogFile / DcGetMaxLogFile

Item	Description
Function prototype	LX_STATE DcSetMaxLogFile(int count); LX_STATE DcGetMaxLogFile(int* count);
Function purpose	Set or obtain the maximum number of rolling log files.

Item	Description
Function parameters	count: number of log files.
Return value and remarks	Used to control log usage for long-running programs.

9.1.10 DcSetMaxLogFileSize / DcGetMaxLogFileSize

Item	Description
Function prototype	LX_STATE DcSetMaxLogFileSize(int size); LX_STATE DcGetMaxLogFileSize(int* size);
Function purpose	Set or obtain the maximum size of a single log file.
Function parameters	size: maximum size of a single log file, in bytes.
Return value and remarks	Used together with the maximum number of log files to control disk usage by logs.

9.2 Camera Discovery, Connection, and Closing

9.2.1 DcGetDeviceList

Item	Description
Function prototype	LX_STATE DcGetDeviceList(LxDeviceInfo** devlist, int* devnum, LX_DEVICE_SERIALS lx_serials = LX_SERIAL_ALL);
Function purpose	Search the supported camera list.
Function parameters	devlist: output device list; devnum: output device quantity; lx_serials: device category filter, default LX_SERIAL_ALL.
Return value and remarks	The device list memory is maintained internally and does not need to be allocated externally. The previous content is cleared before each search. Ensure thread safety when using it in multiple threads. The pure C interface has no default parameter, so the third parameter must be explicitly passed. The sample code uses the C++ form DcGetDeviceList(&p_device_list,&device_num).

9.2.2 DcOpenDevice

Item	Description
Function prototype	LX_STATE DcOpenDevice(LX_OPEN_MODE open_mode, const char* param, DcHandle* handle, LxDeviceInfo* info);
Function purpose	Open a device and obtain exclusive control permission.
Function parameters	open_mode: open mode; param: index/IP/SN/ID. When the SDK runs inside the camera, 127.0.0.1 can be used; handle: output device handle; info: output device information.

Item	Description
Return value and remarks	After opening succeeds, other processes cannot open the same camera again. Subsequent interfaces all depend on handle.

9.2.3 DcCloseDevice

Item	Description
Function prototype	LX_STATE DcCloseDevice(DCHandle handle);
Function purpose	Close the device and release permission and resources.
Function parameters	handle: device handle.
Return value and remarks	Use in pair with DcOpenDevice. It is recommended to stop streaming and unregister callbacks before closing.

9.3 Starting and Stopping Streaming

9.3.1 DcStartStream

Item	Description
Function prototype	LX_STATE DcStartStream(DCHandle handle);
Function purpose	Start the data stream.
Function parameters	handle: device handle.
Return value and remarks	Complete configurations such as stream switches, ROI, Binning, trigger mode, and synchronized frame before starting streaming.

9.3.2 DcStopStream

Item	Description
Function prototype	LX_STATE DcStopStream(DCHandle handle);
Function purpose	Stop the data stream.
Function parameters	handle: device handle.
Return value and remarks	After stopping streaming, some parameters that must be configured before streaming can be reconfigured, or the device can be closed.

9.4 Reading and Setting Camera Parameters

9.4.1 DcSetIntValue

Item	Description
Function prototype	LX_STATE DcSetIntValue(DCHandle handle, int cmd, int value);
Function purpose	Set an int parameter.

Item	Description
Function parameters	handle: device handle; cmd: LX_INT_* in LX_CAMERA_FEATURE; value: value to set.
Return value and remarks	Check whether the parameter is supported and whether the setting timing is valid first.

9.4.2 DcGetIntValue

Item	Description
Function prototype	LX_STATE DcGetIntValue(DCHandle handle, int cmd, LxIntValueInfo* value);
Function purpose	Obtain int parameter information.
Function parameters	value: output set_available, cur_value, max_value, and min_value.
Return value and remarks	Image size, data type, read-only status, and other information are obtained through this interface.

9.4.3 DcSetFloatValue

Item	Description
Function prototype	LX_STATE DcSetFloatValue(DCHandle handle, int cmd, float value);
Function purpose	Set a float parameter.
Function parameters	cmd: LX_FLOAT_*; value: value to set.
Return value and remarks	Used for floating-point parameters such as filtering, light intensity, and confidence.

9.4.4 DcGetFloatValue

Item	Description
Function prototype	LX_STATE DcGetFloatValue(DCHandle handle, int cmd, LxFloatValueInfo* value);
Function purpose	Obtain float parameter information.
Function parameters	value: output set_available, cur_value, max_value, and min_value.
Return value and remarks	Read-only floating-point parameters such as frame rate and temperature are obtained through this interface.

9.4.5 DcSetBoolValue

Item	Description
Function prototype	LX_STATE DcSetBoolValue(DCHandle handle, int cmd, bool value);
Function purpose	Set a bool parameter.
Function parameters	cmd: LX_BOOL_*; value: true or false.

Item	Description
Return value and remarks	Used for switches such as data stream, auto exposure, undistortion, synchronized frame, and IMU.

9.4.6 DcGetBoolValue

Item	Description
Function prototype	LX_STATE DcGetBoolValue(DcHandle handle, int cmd, bool* value);
Function purpose	Obtain a bool parameter.
Function parameters	value: output parameter value.
Return value and remarks	Can read connection status and current enable status.

9.4.7 DcSetStringValue

Item	Description
Function prototype	LX_STATE DcSetStringValue(DcHandle handle, int cmd, const char* value);
Function purpose	Set a string parameter.
Function parameters	cmd: LX_STRING_*; value: string content.
Return value and remarks	Used for firmware file name, algorithm parameter JSON, parameter import/export paths, and custom ID.

9.4.8 DcGetStringValue

Item	Description
Function prototype	LX_STATE DcGetStringValue(DcHandle handle, int cmd, char** value);
Function purpose	Obtain a string parameter.
Function parameters	value: output string pointer. No external allocation is required.
Return value and remarks	Returns internal memory. The next related call may refresh it.

9.4.9 DcGetPtrValue

Item	Description
Function prototype	LX_STATE DcGetPtrValue(DcHandle handle, int cmd, void** value);
Function purpose	Obtain a pointer-type parameter.
Function parameters	cmd: LX_PTR_*; value: output internal data pointer.
Return value and remarks	In synchronous streaming, LX_CMD_GET_NEW_FRAME is usually required before reading pointers. In callbacks, FrameInfo data can be used directly.

9.4.10 DcSetCmd

Item	Description
Function prototype	LX_STATE DcSetCmd(DcHandle handle, int cmd);
Function purpose	Execute a command-type instruction.
Function parameters	cmd:LX_CMD_.*.
Return value and remarks	Common commands include LX_CMD_GET_NEW_FRAME, LX_CMD_GET_NEW_FRAME_3D, LX_CMD_GET_NEW_FRAME_2D, LX_CMD_SOFTWARE_TRIGGER, LX_CMD_RESTART_DEVICE, and LX_CMD_RESET_PARAM.

9.5 Special Control, ROI, IP, User Data, and Point Cloud Saving

9.5.1 DcSpecialControl

Item	Description
Function prototype	LX_STATE DcSpecialControl(DcHandle handle, const char* command, void* value);
Function purpose	Execute special operations outside LX_CAMERA_FEATURE.
Function parameters	command: string command; value: input parameter when setting and output parameter when getting.
Return value and remarks	Common commands include SetObstacleIndex, GetObstacleIndex, SetObstacleMode, GetObstacleIO, SetOdomData, SetRelocPose, SetLaserData, ImportLocationMapFile, EnableBuildMap, EnableLocation, and ExportLocationMapFile.

9.5.2 DcSpecialControlExtend

Item	Description
Function prototype	LX_STATE DcSpecialControlExtend(DcHandle handle, const char* command, void* value, int timeout_ms);
Function purpose	Special control interface with a timeout.
Function parameters	timeout_ms: control timeout, in ms. If it is less than 1, the default value of LX_INT_GVCP_TIME_OUT is used.
Return value and remarks	Can be used in scenarios that require more explicit control of wait time, such as algorithm control and map import/export.

9.5.3 DcSetROI

Item	Description
Function prototype	LX_STATE DcSetROI(DcHandle handle, int offsetx, int offsety, int width, int height, int img_type);

Item	Description
Function purpose	Set the ROI region.
Function parameters	offsetx/offsety: start point offset; width/height: ROI width and height; img_type: 0 for 3D image, 1 for 2D image.
Return value and remarks	If the input value is not a multiple of 8, it is automatically corrected internally. Re-read the image size after setting.

9.5.4 DcSetCameraIp

Item	Description
Function prototype	LX_STATE DcSetCameraIp(DCHandle handle, const char* ip, const char* netmask = 0, const char* gateway = 0);
Function purpose	Set the camera IP, subnet mask, and gateway.
Function parameters	ip: device IP; netmask: defaults to 255.255.0.0 when empty; gateway: when empty, the last segment of the IP is set to 1 by default.
Return value and remarks	After modification, the device list changes. DcGetDeviceList must be called again.

9.5.5 DcWriteUserData

Item	Description
Function prototype	LX_STATE DcWriteUserData(DCHandle handle, int start_address, char* data, int data_size);
Function purpose	Write user-defined content to the device.
Function parameters	start_address: start address, starting from 0; data: content to write; data_size: write length, maximum 128KB.
Return value and remarks	Confirm that the device supports this capability before writing and avoid overwriting important user area data.

9.5.6 DcReadUserData

Item	Description
Function prototype	LX_STATE DcReadUserData(DCHandle handle, int start_address, char** data, int& data_size);
Function purpose	Read user-defined content from the device.
Function parameters	data: output content pointer; data_size: output length, maximum 128KB.
Return value and remarks	No external memory allocation is required. Before each read, the memory allocated during the previous read is reset. In the current header file, data_size is int&, which is C++ reference syntax. If a pure C project is used, confirm whether the SDK provides an int* version, or compile the code that calls this interface as C++.

9.5.7 DcSaveXYZ

Item	Description
Function prototype	LX_STATE DcSaveXYZ(DcHandle handle, const char* filename);
Function purpose	Save ordinary XYZ point cloud.
Function parameters	filename: file name, supporting txt, ply, and pcd.
Return value and remarks	Must be called in the data acquisition thread. txt saves all data in image order. ply and pcd save only non-zero data.

9.5.8 DcSetParamPath

Item	Description
Function prototype	LX_STATE DcSetParamPath(DcHandle handle, const char* filepath);
Function purpose	Set the parameter file path.
Function parameters	filepath: parameter file path, excluding the file name.
Return value and remarks	In most cases this does not need to be called. Only some models require an external parameter path when opening.

9.6 Callback Interfaces

9.6.1 DcRegisterFrameCallback

Item	Description
Function prototype	LX_STATE DcRegisterFrameCallback(DcHandle handle, LX_FRAME_CALLBACK func, void* usr_data = 0);
Function purpose	Register a data frame callback function. It is called automatically when new data is received.
Function parameters	func: callback function pointer; usr_data: user-defined parameter, which can be null.
Return value and remarks	Do not execute long blocking operations inside the callback.

9.6.2 DcUnregisterFrameCallback

Item	Description
Function prototype	LX_STATE DcUnregisterFrameCallback(DcHandle handle);
Function purpose	Unregister the data frame callback.
Function parameters	handle: device handle.
Return value and remarks	Unregister before closing the device.

9.6.3 DcRegisterCameraStatusCallback

Item	Description
Function prototype	LX_STATE DcRegisterCameraStatusCallback(DCHandle handle, LX_CAMERA_STATUS_CALLBACK func, void* usr_data = 0);
Function purpose	Register a camera status callback function.
Function parameters	func: status callback function; usr_data: user-defined parameter.
Return value and remarks	When the status changes, CameraStatus is returned through the callback. This is used for multi-camera and disconnection/reconnection monitoring.

9.6.4 DcUnregisterCameraStatusCallback

Item	Description
Function prototype	LX_STATE DcUnregisterCameraStatusCallback(DCHandle handle);
Function purpose	Unregister the camera status callback.
Function parameters	handle: device handle.
Return value and remarks	Unregister before closing the device.

9.6.5 DcRegisterImuDataCallback

Item	Description
Function prototype	LX_STATE DcRegisterImuDataCallback(DCHandle handle, LX_IMUDATA_CALLBACK func, void* usr_data);
Function purpose	Register an IMU data callback function. It is called automatically when new IMU data is received.
Function parameters	func: IMU callback function; usr_data: user-defined parameter.
Return value and remarks	Use together with LX_BOOL_ENABLE_IMU to enable IMU.

9.6.6 DcUnregisterImuDataCallback

Item	Description
Function prototype	LX_STATE DcUnregisterImuDataCallback(DCHandle handle);
Function purpose	Unregister the IMU data callback.
Function parameters	handle: device handle.
Return value and remarks	Unregister before closing the device.

10. Sample Program Description

10.1 Sample Program Index

Sample	Path	Key Points to Check
single_camera2	Sample/C/single_camera2/single_camera2.cpp	Single-camera synchronous streaming, trigger, FrameInfo, XYZ point cloud, and image display.
frame_callback	Sample/C/frame_callback/frame_callback.cpp	Asynchronous frame callback, FrameExtendInfo, ordinary point cloud inside the callback, and app_data.
lidarpoint_callback	Sample/C/lidarpoint_callback/lidarpoint_callback.cpp	Obtaining LiDAR XYZIRT point cloud through the frame callback, PCD saving, and synchronized frame.
imu_callback	Sample/C/imu_callback/imu_callback.cpp	IMU switch, range level, IMU callback, and frequency statistics.
multi_cameras	Sample/C/multi_cameras/multi_cameras.cpp	Multiple IPs and multiple threads, status callback, compatible reading of old image pointers, and display thread.
application_obstacle_v1	Sample/C/application_obstacle_v1/application_obstacle_v1.cpp	Obstacle avoidance V1, SetObstacleIndex, GetObstacleIndex, and GetObstacleIO.
application_obstacle_v2	Sample/C/application_obstacle_v2/application_obstacle_v2.cpp	Obstacle avoidance V2, SetObstacleMode, and LxAvoidanceOutputN.
application_pallet	Sample/C/application_pallet/application_pallet.cpp	Pallet docking, algorithm parameter JSON, and LxPalletPose.
application_location	Sample/C/application_location/application_location.cpp	Visual localization, map, odometry, relocalization, laser data, and RGB. It contains test paths. Before actual running, replace them with map, odometry, or log paths that exist on the local machine; otherwise, the sample will fail.

10.2 single_camera2: Single-Camera Synchronous Streaming

Query Item	Description
Startup and connection	Set logs, read the API version, search devices through DcGetDeviceList, and open a device by index or IP.

Query Item	Description
Stream configuration	Enable or disable LX_BOOL_ENABLE_3D_DEPTH_STREAM, LX_BOOL_ENABLE_3D_AMP_STREAM, and LX_BOOL_ENABLE_2D_STREAM. LX_BOOL_ENABLE_SYNC_FRAME can be enabled optionally.
Alignment and trigger	RGBD alignment can be set through LX_INT_RGBD_ALIGN_MODE. Trigger mode uses LX_INT_TRIGGER_MODE, and software trigger uses LX_CMD_SOFTWARE_TRIGGER.
Synchronous frame acquisition	Call LX_CMD_GET_NEW_FRAME. LX_CMD_GET_NEW_FRAME_3D or LX_CMD_GET_NEW_FRAME_2D can also be called as needed.
Data reading	Obtain FrameInfo through LX_PTR_FRAME_DATA. The sample processes depth_data, amp_data, and rgb_data separately. Convert reserve_data to FrameExtendInfo to read depth, amp, and rgb frame IDs. This sample does not parse app_data.
Point cloud	Obtain ordinary XYZ point cloud through LX_PTR_XYZ_DATA, or use DcSaveXYZ to save txt, ply, or pcd. LX_PTR_XYZIRT_DATA is used for LiDAR XYZIRT point cloud. See the lidarpoint_callback sample.

10.3 frame_callback: Frame Callback Streaming

Query Item	Description
Register callback	Call DcRegisterFrameCallback(handle, callback, user_data), and then start streaming.
Callback data	The callback input parameter is FrameInfo*. Check frame_state directly and read depth_data, amp_data, rgb_data, and app_data.
Extended information	reserve_data can be converted to FrameExtendInfo to obtain depth, amp, rgb, and app frame IDs. Point cloud acquisition In the callback context, DcGetPtrValue(LX_PTR_XYZ_DATA) can be called to obtain ordinary point cloud. LiDAR XYZIRT point cloud is obtained through LX_PTR_XYZIRT_DATA. See the lidarpoint_callback sample.
Point cloud acquisition	In the callback context, DcGetPtrValue(LX_PTR_XYZ_DATA) can be called to obtain ordinary point cloud. LiDAR XYZIRT point cloud is obtained through LX_PTR_XYZIRT_DATA. See the lidarpoint_callback sample.
Exit handling	Unregister DcUnregisterFrameCallback before stopping streaming to avoid callbacks continuing to access released resources.

10.4 lidarpoint_callback: Obtaining LiDAR Point Cloud Through Frame Callback

Query Item	Description
Synchronization requirement	The sample enables LX_BOOL_ENABLE_SYNC_FRAME to keep the relationship between the output structure and frames more stable.
Point cloud interface	Obtain LxPointCloudData through LX_PTR_XYZIRT_DATA. It includes timebase, point_num, and points, and can be saved as a PCD file.
Point fields	LxPointXYZIRT includes x, y, z, intensity, offset_time, roi_index, row_pos, col_pos, and tag.
PCD saving	The sample saves ASCII PCD fields x, y, z, intensity, and time, but the source code writes point.offset_time to the time column. If absolute time is required, calculate it from pointcloud_data->timebase + point.offset_time.
Note	After 3D undistortion or RGBD alignment is enabled, timestamps, ROI indexes, and row/column positions in the LiDAR point cloud change accordingly.

10.5 imu_callback: IMU Data Callback

Query Item	Description
Call workflow	The sample first calls DcRegisterImuDataCallback to register the IMU callback, then disables the 2D, depth, and amplitude streams. It then enables IMU through DcSetBoolValue(handle,LX_BOOL_ENABLE_IMU,true), sets the ranges through LX_INT_IMU_ACCELERATION_LEVEL and LX_INT_IMU_ANGULAR_RANGE_LEVEL, and finally calls DcStartStream to start receiving IMU data.
Data units	In LxImuSinkData, acc_x/y/z is in m/s ² , and gry_x/y/z is in rad/s.
Frequency statistics	The sample calculates the average IMU frequency based on intervals between multiple callbacks to confirm whether the data is stable.

10.6 multi_cameras: Multi-Camera Development

Query Item	Description
Open mode	The sample opens devices by IP. Each IP uses an independent thread and an independent DCHandle.
Status callback	DcRegisterCameraStatusCallback is used to register a status callback and output statuses such as STATUS_CLOSED, STATUS_STARTED, and STATUS_CONNECTING.
Streaming mode	The sample uses LX_CMD_GET_NEW_FRAME for synchronous refresh, and then reads depth and 2D images. The amplitude stream is disabled in the sample, so amplitude images are not read.

Query Item	Description
Compatible pointers	The sample still uses compatible pointers such as LX_PTR_3D_DEPTH_DATA and LX_PTR_2D_IMAGE_DATA. For new projects, unified reading through LX_PTR_FRAME_DATA is recommended.
Display thread	OpenCV display is placed in a separate thread or queue to avoid blocking the acquisition thread.

10.7 application_obstacle_v1: Obstacle Avoidance Algorithm V1

Query Item	Description
Algorithm mode	Set LX_INT_ALGORITHM_MODE to MODE_AVOID_OBSTACLE.
Stream configuration	The sample starts the algorithm stream after disabling ordinary image streams, and reads the algorithm version and parameters.
Region switching	SetObstacleIndex is used to quickly switch the current obstacle avoidance region index. GetObstacleIndex is used to query the current index.
IO query	GetObstacleIO periodically queries the obstacle avoidance IO output status.
Mode setting	SetObstacleMode can set the complete obstacle avoidance mode, but switching is usually slower than SetObstacleIndex.

10.8 application_obstacle_v2: Obstacle Avoidance Algorithm V2

Query Item	Description
Algorithm mode	Set LX_INT_ALGORITHM_MODE to MODE_AVOID_OBSTACLE2.
Timeout configuration	LX_INT_GVCP_TIME_OUT can be set as needed to control device command timeout.
Mode setting	Use SetObstacleMode to set the obstacle avoidance V2 mode.
Frame acquisition requirement	LX_CMD_GET_NEW_FRAME must be called before reading images or detailed algorithm output.
Output structure	Obtain LxAvoidanceOutputN through LX_PTR_ALGORITHM_OUTPUT. When state is LxAvSuccess, parse groundPlane, cloud_output, and obstacleBoxes
Obstacle information	LxObstacleBoxN includes 3D Box, 2D box ratio, semantic category, ID, historical ID, and other information.

10.9 application_pallet: Pallet Docking Algorithm

Query Item	Description
Algorithm mode	Set LX_INT_ALGORITHM_MODE to MODE_PALLET_LOCATE.
Parameter configuration	The algorithm version and parameters can be read. When customization is required, write JSON through LX_STRING_ALGORITHM_PARAMS.

Query Item	Description
Work mode	The sample can set LX_INT_WORK_MODE to always-on mode to match the application scenario.
Frame acquisition output	After LX_CMD_GET_NEW_FRAME is called, LxPalletPose is read through LX_PTR_ALGORITHM_OUTPUT.
Result interpretation	return_val indicates the result status. x, y, and yaw indicate the pallet distance, lateral offset, and angle relative to the camera. extents stores extended information.

10.10 application_location: Visual Localization Algorithm

Query Item	Description
Algorithm mode	Set LX_INT_ALGORITHM_MODE to MODE_VISION_LOCATION.
Map control	Use ImportLocationMapFile, EnableBuildMap, EnableLocation, and ExportLocationMapFile to manage map import, mapping, localization, and export.
Parameter JSON	Set localization parameters such as map_name and external_param through LX_STRING_ALGORITHM_PARAMS.
Odometry	The sample reads pose.log and periodically sends LxOdomData through SetOdomData. In the initial stage, SetRelocPose can be called to set the relocalization pose.
Laser input	SetLaserData uses LxLaser. ranges is allocated by the user and released after sending.
Localization output	Obtain LxLocation through LX_PTR_ALGORITHM_OUTPUT and output timestamp, status, x, y, and theta to the log file.
RGB data	After enabling the 2D stream, the sample reads the 2D image width, height, channel, data type, and LX_FLOAT_2D_IMAGE_FPS.

11. Debugging and FAQs

Table 11-1 Common Troubleshooting

Symptom	Items to Check First
DcGetDeviceList cannot find the device	Confirm camera power supply, network cable, host network port IP subnet, firewall, network port negotiated speed, and whether there is a cross-subnet routing problem.
DcOpenDevice returns a permission error	Confirm that no other process, or a process left by the previous abnormal exit, is occupying the device. Retry after the heartbeat timeout.
Streaming fails	Confirm that at least one data stream is enabled. Check LX_E_ENABLE_ANYSTREAM_FAILED, network bandwidth, and whether the device supports the selected stream.

Symptom	Items to Check First
Synchronous frame acquisition times out	Check whether DcStartStream has been called, whether software trigger or external trigger is executed in trigger mode, and adjust LX_INT_GVCP_TIME_OUT if necessary.
FrameInfo.frame_state is not successful	Use the status code to determine whether the issue is frame loss, unsynchronization, network error, image parsing error, or multi-camera interference.
Point cloud is empty	Confirm that the depth stream is enabled and the current frame is valid. Check minimum/maximum depth, amplitude threshold, filtering, and scene distance.
RGBD alignment result is abnormal	Confirm the LX_INT_RGBD_ALIGN_MODE setting, 2D/3D stream status, and whether size and intrinsic parameters are re-read after resolution changes.
IMU has no callback	Confirm that the device supports IMU, LX_BOOL_ENABLE_IMU is set, DcRegisterImuDataCallback is registered, and the device has started.
Algorithm output is empty	Confirm that LX_INT_ALGORITHM_MODE has been set, algorithm parameters are valid, streaming has started, and frames have been refreshed. Check the status field before reading the structure.
Multi-camera display stutters	Reduce unnecessary data streams, lower RGB output bandwidth, and avoid time-consuming display or disk writing in the acquisition thread.
Connection fails after IP modification	Search the device list again, confirm that the host network port has switched to the new subnet, and check the gateway and subnet mask if necessary.

Table 11-2 Recommended Log Information

Scenario	Recommended Records
Program startup	API version, log path, whether OpenCV is enabled, GPU/PTP/parallel thread/JPEG decoding configuration.
Device discovery	Device quantity, IP, SN, MAC, device model, firmware version, and algorithm version.
Open device	Open mode, param, return value, handle, and LxDeviceInfo.
Parameter configuration	Feature name, target value, return value, and necessary readback for each setting item.
Frame acquisition	DcSetCmd return value, FrameInfo.frame_state, frame ID, timestamp, image size, and data type.
Algorithm	Algorithm mode, algorithm version, parameter JSON, special control command, and output structure status.

12. Version and Compatibility Notes

Table 12-1 Version Information

Version	Description
2.4.60	Current header file version macro. New and retained content includes IMU callback, LiDAR point cloud, RGBD alignment enumeration, more device models, user data read/write, GPU/PTP/log rolling, extended special control, and others.
2.4.16	Supports S2MAX cameras, adds parameter import/export and parameter group switching, and adds related function settings.
2.4.9	Optimizes and adds some functions and configuration parameters. Timestamp unit changed to us.
2.4.2	Supports M4, M4Mega, M4Pro, V1Pro, S2, H3, and other cameras.
2.0.2	Fixes some known issues and adjusts some interfaces and parameters.

Table 12-2 Compatibility and Recommended Usage

Historical Compatibility Item	Recommended or Current Usage	Description
DcSetRoI	DcSetROI	The current header file interface name is DcSetROI.
LX_BOOL_ENABLE_2D_TO_DEPTH	LX_INT_RGBD_ALIGN_MODE	Use RGBD_ALIGN_OFF, DEPTH_TO_RGB, RGB_TO_DEPTH, or DEPTH_TO_RGB_SPARSE to explicitly specify the alignment mode.
LX_PTR_2D_IMAGE_DATA	LX_PTR_FRAME_DATA.rgb_data	The separate image pointer remains compatible. For new projects, unified reading through FrameInfo is recommended.
LX_PTR_3D_DEPTH_DATA	LX_PTR_FRAME_DATA.depth_data	Unified reading can obtain data type, size, channel, and timestamp at the same time.
LX_PTR_3D_AMP_DATA	LX_PTR_FRAME_DATA.amplitude_data	Unified reading is convenient for synchronized processing of depth, amplitude, and RGB.
LX_PTR_ALGORITHM_OUTPUT	FrameInfo.app_data or compatible direct reading	The algorithm synchronization samples still use this pointer directly. For frame callback scenarios, reading from app_data is recommended.
LX_PTR_2D_INTRINSIC_PARAMETERS / LX_PTR_3D_INTRINSIC_PARAMETERS	LX_PTR_2D_INTRINSIC_PARAMETERS / LX_PTR_3D_INTRINSIC_PARAMETERS	The new interface returns LxIntrinsicParameters, including resolution, intrinsic matrix, distortion model, and number of distortion coefficients.
LX_FLOAT_2D_IMAGE_FPS	LX_FLOAT_2D_IMAGE_FPS	The current header file name is LX_FLOAT_2D_IMAGE_FPS.

Historical Compatibility Item	Recommended or Current Usage	Description
LX_BOOL_ENABLE_HDR	LX_BOOL_ENABLE_MULTI_EXPOSURE_HDR	The current header file uses the multi-exposure HDR switch.
LX_BOOL_ENABLE_TOF_DEPTH_CALIBE, LX_INT_TOF_FREQMODE	Not defined as available Features in the current header file	New projects should not call them as current SDK parameters.